

Play4Change – Aplicação de Aprendizagem Adaptativa para Educação Cívica

Radesh Ilesh Gamanbhai Govind

Orientadores: Nuno Datia
Michel Vorenhout
António Serrador

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

11 de Julho de 2026

INSTITUTO POLITÉCNICO DE LISBOA

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

**Play4Change – Aplicação de Aprendizagem Adaptativa para Educação
Cívica**

A51620 Radesh Ilesh Gamanbhai Govind

Orientadores: Nuno Datia

Michel Vorenhout

António Serrador

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

11 de Julho de 2026

Resumo

Apesar de o referencial europeu DigComp 2.2 (*Digital Competence Framework for Citizens*) estruturar as competências digitais essenciais à cidadania e servir de base a políticas de literacia digital em toda a Europa, escasseiam plataformas de aprendizagem que recorram a Inteligência Artificial (IA) para personalizar o percurso de cada utilizador e que gerem e adaptem autonomamente o seu próprio conteúdo pedagógico.

Esta lacuna é particularmente relevante à luz do “problema dos 2 sigma” de Bloom (a tutoria individual supera claramente o ensino tradicional em sala de aula, mas é historicamente inviável a larga escala), que os *Large Language Models* (LLM) tornam hoje possível aproximar a um custo marginal reduzido. O projeto insere-se na aliança europeia U!REKA SHIFT (Instituto Politécnico de Lisboa/ISEL e Amsterdam University of Applied Sciences), alinhando-se com a meta europeia de dotar a população de competências digitais básicas até 2030.

O Play4Change responde a esta lacuna com uma plataforma de aprendizagem adaptativa composta por um painel de administração web, uma aplicação móvel centrada num desafio diário e um servidor Spring Boot. A plataforma gera automaticamente conteúdo pedagógico por Inteligência Artificial a partir de documentos fornecidos por um formador; quando um Learner erra, o sistema adapta-se às dificuldades de cada um e se necessário ainda escala para tutoria conversacional com IA.

Uma avaliação de usabilidade com utilizadores reais registou uma pontuação SUS (*System Usability Scale*) média de 76,0, acima da média de referência da indústria (68,0), com a tutoria por IA identificada como a funcionalidade mais valorizada, um resultado que atesta a facilidade de uso, não o ganho de competências DigComp 2.2. Os resultados confirmam a viabilidade técnica de uma plataforma que gera e adapta autonomamente o seu conteúdo pedagógico; a avaliação do ganho pedagógico a larga escala fica para trabalho futuro.

Palavras-chave: Aprendizagem Adaptativa; Inteligência Artificial Generativa; *Retrieval-Augmented Generation*; Competências Digitais; DigComp 2.2.

Abstract

Although the European DigComp 2.2 (*Digital Competence Framework for Citizens*) framework structures the digital competences essential to citizenship and underpins digital literacy policy across Europe, adaptive learning platforms that leverage Artificial Intelligence (AI) to personalise each user’s learning path, and that autonomously generate and adapt their own pedagogical content remain scarce.

This gap is particularly relevant in light of Bloom’s “2 sigma problem” (one-to-one tutoring clearly outperforms traditional classroom teaching, but has historically been unaffordable at scale), which *Large Language Models* (LLMs) now make achievable at a fraction of the cost. The project is part of the European U!REKA SHIFT alliance (Instituto Politécnico de Lisboa/ISEL and the Amsterdam University of Applied Sciences), aligning with the EU goal of equipping the population with basic digital competences by 2030.

Play4Change addresses this gap with an adaptive learning platform comprising a web administration panel, a mobile application centred on a daily challenge, and a Spring Boot server. The platform automatically generates pedagogical content via Artificial Intelligence from documents supplied by an instructor; when a Learner answers incorrectly, the system adapts to each user’s difficulties and, if necessary, further escalates to conversational AI tutoring.

A usability evaluation with real users recorded an average System Usability Scale (SUS) score of 76.0, above the industry reference average (68.0), with AI tutoring identified as the most valued feature, a result that attests to ease of use, not to DigComp 2.2 competence gains. The results confirm the technical feasibility of a platform that autonomously generates and adapts its own pedagogical content; evaluating pedagogical gains at scale is left for future work.

Keywords: Adaptive Learning; Generative Artificial Intelligence; Retrieval-Augmented Generation; Digital Competences; DigComp 2.2.

Utilização de Inteligência Artificial

No desenvolvimento deste projeto foram utilizadas as seguintes ferramentas de inteligência artificial generativa:

- **Google Gemini Flash e Gemini Pro:** utilizados principalmente na fase de investigação e enquadramento: pesquisa de conceitos, leitura e síntese de documentação técnica, e exploração de alternativas de design antes de tomar decisões de arquitetura. Foram também utilizados para *debater* e estudar tópicos novos, funcionando como interlocutor para clarificar dúvidas sobre padrões de domínio e comportamento de APIs externas.
- **Claude Sonnet (Claude Code e Claude Cowork):** utilizados ao longo de todo o ciclo de desenvolvimento. O Claude Code (interface de linha de comandos) foi o principal auxiliar de programação: geração de esqueletos de código, sugestão e implementação de refactoros, diagnóstico e correção de bugs, e escrita de testes unitários. O Claude Cowork (interface de desktop) foi utilizado na fase de redação do relatório: análise de saídas do Claude Code, geração de figuras de diagramas de sequência com matplotlib.
- **Prova de conceitos (*proof of concepts*):** em várias ocasiões, as ferramentas de IA foram utilizadas para construir protótipos mínimos que permitiram validar a viabilidade técnica de uma abordagem antes de a integrar no projeto principal.

Em todos os casos, o código e o texto gerados foram revistos, iterados, adaptados e validados pelo autor.

Lista de Acrónimos e Siglas

Sigla	Significado
API	<i>Application Programming Interface</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CSP	<i>Content Security Policy</i>
DAG	<i>Directed Acyclic Graph</i> (grafo acíclico dirigido)
DDD	<i>Domain-Driven Design</i>
DigComp	<i>Digital Competence Framework for Citizens</i>
DNS	<i>Domain Name System</i>
ECTS	<i>European Credit Transfer and Accumulation System</i>
HTTP(S)	<i>HyperText Transfer Protocol (Secure)</i>
IA	Inteligência Artificial
ITS	<i>Intelligent Tutoring System</i>
JPA	<i>Jakarta Persistence API</i>
JSON	<i>JavaScript Object Notation</i>
JVM	<i>Java Virtual Machine</i>
JWT	<i>JSON Web Token</i>
LLM	<i>Large Language Model</i>
MVI	<i>Model-View-Intent</i>
OCR	<i>Optical Character Recognition</i>
OWASP	<i>Open Worldwide Application Security Project</i>
RAG	<i>Retrieval-Augmented Generation</i>
RBAC	<i>Role-Based Access Control</i>
REST	<i>Representational State Transfer</i>
SPA	<i>Single Page Application</i>
SQL	<i>Structured Query Language</i>
SSE	<i>Server-Sent Events</i>
SSRF	<i>Server-Side Request Forgery</i>
SUS	<i>System Usability Scale</i>
TOCTOU	<i>Time-Of-Check to Time-Of-Use</i>
TTL	<i>Time To Live</i>
URL	<i>Uniform Resource Locator</i>
UUID	<i>Universally Unique Identifier</i>
XSS	<i>Cross-Site Scripting</i>

Índice

1	Introdução	1
1.1	Contextualização	1
1.2	Motivação	1
1.3	Objetivos	2
1.4	Âmbito, Limitações e Organização do Documento	2
2	Estado da Arte	3
2.1	Aprendizagem Adaptativa e Sistemas de Tutoria Inteligente	3
2.2	Inteligência Artificial na Educação	4
2.3	Aprendizagem Formal, Informal e Móvel	4
2.4	Retrieval-Augmented Generation	5
2.5	Síntese Comparativa	6
3	Análise e Requisitos	7
3.1	Identificação dos Utilizadores	7
3.1.1	Administrador	7
3.1.2	Learner	7
3.2	Requisitos Funcionais	8
3.3	Requisitos Não Funcionais	9
3.4	Casos de Uso	9
3.4.1	CU01: Submeter Conteúdo e Gerar Desafios	10
3.4.2	CU02: Responder ao Desafio Diário	10
3.4.3	CU03: Remediação Adaptativa	11
4	Arquitetura e Fundações	13
4.1	Arquitetura Geral do Sistema	13
5	Implementação	15
5.1	Fundações: Servidor, Pipeline e Autenticação	15
5.1.1	Pipeline de Processamento de Pedidos	16
5.1.2	Autenticação sem Password	17

5.2	Do Conteúdo ao Desafio	19
5.2.1	Criação do Conteúdo	19
5.2.2	Consumo do Conteúdo	23
5.2.2.1	Atribuição Diária	23
5.2.2.2	Remediação Adaptativa	24
5.2.2.3	Tutoria com IA	26
5.3	Segurança: Cinco Camadas de Defesa	27
6	Testes	31
6.1	Testes Automatizados	31
6.2	Testes de Usabilidade	32
6.2.1	Metodologia	32
6.2.2	Análise SUS	33
6.2.3	Análise Qualitativa	34
7	Discussão	35
7.1	Objetivos Atingidos	35
7.2	O que Correu Bem	36
7.3	Limitações e Dificuldades	36
7.4	Comparação com o Estado da Arte	37
7.5	Trabalho Futuro	38
	Referências	41
A	Exemplos de Prompts Completos	43
A.1	Prompt de Geração de Tópicos e Desafios	43
A.2	Prompt de Análise de Dificuldade e Geração de Subtarefas Adaptativas	45
A.3	Prompt de Explicação Holística e Conversação	47

Capítulo 1

Introdução

A transformação digital acelerada das últimas décadas tornou a capacidade de utilizar as tecnologias digitais de forma confiante, crítica e responsável uma competência fundamental para a vida moderna. A União Europeia reconheceu esta necessidade através do *Digital Competence Framework for Citizens* (DigComp), atualmente na versão 2.2, que organiza 21 competências digitais em cinco áreas e serve de base a políticas de educação digital em toda a Europa [1]. Este projeto insere-se na aliança universitária europeia U!REKA, da qual o IPL é membro ativo, e é desenvolvido em colaboração com a Amsterdam University of Applied Sciences (HvA), parceira da rede.

1.1 Contextualização

O *Digital Compass* e o subsequente *Digital Decade Policy Programme* da União Europeia fixam o objetivo de dotar pelo menos 80% da população europeia de competências digitais básicas até 2030, tendo o DigComp 2.2 como instrumento de referência para o operacionalizar [2, 1]. É neste sentido que o presente projeto entende a educação cívica: as competências digitais do DigComp 2.2 constituem hoje uma dimensão essencial da cidadania: utilizar tecnologia de forma crítica, segura e responsável é participar plenamente na vida cívica digital. Apesar deste enquadramento, escasseiam plataformas de aprendizagem que tirem partido da Inteligência Artificial (IA) para personalizar a experiência de aprendizagem, e que permitam a gestão autónoma de conteúdos adaptados ao perfil e progresso de cada utilizador.

1.2 Motivação

O “problema dos 2 sigma” de Bloom (a tutoria individual supera em dois desvios padrão o ensino tradicional em sala de aula, mas é economicamente inviável a larga escala [3]) é discutido em detalhe na Secção 2.2; os LLM reabriram este debate ao aproximar esse ideal do custo marginal [4]. No domínio específico do DigComp 2.2 não existe, até ao momento, uma plataforma adaptativa que recorra à IA para personalizar o percurso de cada utilizador.

Técnicas como o *Retrieval-Augmented Generation* (RAG) permitem ancorar as respostas do modelo em conhecimento específico e verificável [5], abrindo a oportunidade de conjugar o quadro estruturado do DigComp 2.2 com uma experiência na interseção entre aprendizagem formal e informal [6].

1.3 Objetivos

O objetivo geral deste projeto é construir o **Play4Change**: um painel de administração web, um cliente móvel para Learners, e um sistema que recorre a Inteligência Artificial (IA) para gerar e adaptar pedagogicamente o conteúdo educativo entregue nesse cliente.

Aprofundando este objetivo, a plataforma centra-se no desenvolvimento de competências digitais, servindo dois perfis distintos: os **Administradores**, que gerem e criam conteúdos através do painel web, e os **Learners**, que desenvolvem as suas competências através da aplicação móvel, assente no modelo de um desafio diário.

O projeto concretiza-se em cinco objetivos operacionais, detalhados individualmente no Capítulo 7 (§7.1).

1.4 Âmbito, Limitações e Organização do Documento

O Play4Change foi desenvolvido no âmbito do projeto *U!REKA SHIFT*, iniciativa da aliança U!REKA que visa soluções educativas inovadoras para as cidades do futuro, com foco na literacia digital. A colaboração entre o IPL e a HvA enquadra-se neste objetivo, contribuindo para as metas do *Digital Compass* para 2030 [2]. Em termos técnicos, o âmbito cobre o *backend*, o cliente web e o cliente móvel para Learners.

As principais limitações são: a avaliação pedagógica da eficácia da plataforma a larga escala não foi realizada, sendo o foco a conceção técnica do sistema; a integração com sistemas de gestão de aprendizagem externos (LMS) fica para trabalho futuro; e, apesar de a arquitetura Kotlin Multiplatform permitir compilação nativa para iOS, o projeto entrega apenas o alvo Android.

O restante relatório organiza-se do geral para o específico, seguindo a mesma lógica desta introdução: o Capítulo 2 situa o Play4Change no estado da arte da aprendizagem adaptativa, da IA na educação e do RAG; o Capítulo 3 traduz essa análise em requisitos e casos de uso concretos; o Capítulo 4 apresenta a arquitetura que responde a esses requisitos; o Capítulo 5 detalha a implementação dessa arquitetura, do conteúdo submetido pelo Administrador ao desafio entregue ao Learner, com destaque para as decisões mais distintivas do sistema; o Capítulo 6 valida essa implementação em duas frentes, a suite de testes automatizados e a avaliação de usabilidade; e o Capítulo 7 discute os resultados face aos objetivos e ao estado da arte.

Capítulo 2

Estado da Arte

Este capítulo situa o Play4Change no panorama da aprendizagem adaptativa, da Inteligência Artificial na educação, da distinção entre aprendizagem formal, informal e móvel, e do *Retrieval-Augmented Generation*, terminando com uma síntese comparativa face às soluções existentes.

2.1 Aprendizagem Adaptativa e Sistemas de Tutoria Inteligente

A ideia de adaptar o ensino ao ritmo e ao perfil individual de cada aluno não é nova: em meados do século XX, Skinner propôs as primeiras “máquinas de ensinar”, baseadas no reforço imediato da resposta correta [7], evoluindo nas décadas seguintes para os *Intelligent Tutoring Systems* (ITS), sistemas que modelam o estado de conhecimento do aluno e adaptam dinamicamente a instrução através de três módulos (domínio, estudante e pedagógico) [8, 9], o segundo tipicamente implementado por *Knowledge Tracing* [10, 11].

No mercado atual, plataformas como Duolingo, Khan Academy e Coursera incorporam adaptatividade em graus distintos e têm vindo, desde 2023–2024, a recorrer também elas a Inteligência Artificial generativa para acelerar a produção de conteúdo: a Duolingo lançou 148 novos cursos de línguas construídos com IA generativa [12]; a Khan Academy oferece, através do Khanmigo, geração dinâmica de planos de aula, questões e diferenciação por interesse do aluno [13]; e a Coursera disponibiliza aos seus parceiros um Course Builder que gera automaticamente estrutura, leituras e avaliações a partir de indicações do autor [14]. A limitação que persiste não é a geração de conteúdo em si, mas o seu âmbito: em todos os casos a IA alimenta o catálogo próprio e curado da plataforma, não a submissão livre de uma fonte arbitrária por qualquer Administrador. É nesse âmbito que o Play4Change se distingue: a ingestão de uma fonte própria do Administrador, a geração ancorada por RAG e a remediação adaptativa por Learner, integradas no mesmo ciclo.

2.2 Inteligência Artificial na Educação

A aplicação da Inteligência Artificial à educação (*AI in Education*, AIED) tem uma história de várias décadas, mas os LLM introduziram uma mudança qualitativa nas possibilidades desta área. Em 1984, Bloom demonstrou que a tutoria individual supera em dois desvios padrão (2σ) o ensino tradicional em sala de aula, um ganho pedagógico economicamente inviável a essa escala [3]. Os LLM reabriram este debate [4]: um modelo disponível 24 horas, capaz de adaptar a explicação ao nível do utilizador e dar *feedback* imediato e contextualizado, aproxima pela primeira vez esse ideal de uma escala de implementação massiva [15].

Estar disponível a qualquer hora não basta, contudo, para que essa tutoria seja pedagogicamente eficaz, e é aqui que a literatura aponta o primeiro requisito que um tutor por IA precisa de cumprir. A investigação converge num ponto, independentemente da tecnologia que a entrega: o *feedback* imediato e formativo acelera a aprendizagem. Hattie (2009) coloca-o entre os dez maiores aceleradores, com um tamanho de efeito $d = 0,73$ [16], ainda que a robustez estatística desta meta-análise tenha sido questionada [17]. É este o padrão que o Play4Change persegue, ainda que de forma parcial: fornece de imediato o resultado (certo/errado) de cada desafio diário, mas o *feedback* explicativo por IA só é acionado quando o Learner erra e a remediação adaptativa subsequente também falha (Secção 5.2.2.2 do Capítulo 5). Uma escalada deliberada, não a resposta imediata e exaustiva que a tecnologia já permitiria.

Um tutor sempre disponível e com *feedback* imediato traz consigo um segundo risco, este do lado oposto: o de sobrecarregar o aluno pela própria frequência da interação. A *Cognitive Load Theory* de Sweller [18] distingue carga cognitiva *intrínseca* (a dificuldade inerente ao próprio conteúdo a aprender), *extrínseca* (a dificuldade acrescida pela forma como esse conteúdo é apresentado, evitável por bom desenho instrucional) e *germane* (o esforço mental produtivo que o aluno dedica a consolidar o que aprende), e é este o segundo requisito que orienta o desenho do Play4Change: aplicado à escala do tópico, traduz-se num único desafio curto e diário por tópico, sem conteúdo simultâneo a competir pela atenção do Learner dentro desse tópico. É também essa unidade por tópico que permite a um Administrador ativar um novo domínio de conhecimento sem redesenhar a experiência dos restantes. A carga cognitiva agregada só cresce, ainda assim, por opção do próprio Learner: ao inscrever-se em vários tópicos em simultâneo (Secção 5.2.2.1 do Capítulo 5), cada tópico ativo soma um desafio diário independente, um comportamento que a versão atual não limita nem orienta explicitamente.

2.3 Aprendizagem Formal, Informal e Móvel

A distinção entre aprendizagem formal e informal é central para compreender o posicionamento do Play4Change. A **aprendizagem formal** caracteriza-se por ser institucionalizada,

estruturada e avalizada por certificações reconhecidas, com motivação frequentemente extrínseca [19]. A **aprendizagem informal**, por contraste, é não estruturada, autodirigida e motivada pela necessidade imediata ou curiosidade intrínseca [6]: Knowles demonstrou que os adultos aprendem melhor quando a aprendizagem resolve problemas reais e é controlada pelo próprio [20], e Cross popularizou a estimativa (originada em investigação anterior do Institute for Research on Learning) de que 80% do conhecimento prático usado no trabalho é adquirido informalmente [6]. O Play4Change opera deliberadamente na interseção destes dois paradigmas: ancorado no referencial estruturado do DigComp 2.2, mas com um modelo de utilização informal e autodirigido, sem avaliações sumativas. A única concessão a um horário é o próprio desafio diário: o Learner escolhe quando o resolver dentro de uma janela de 24 horas, sujeita a uma penalização progressiva por atraso em vez de um prazo rígido de submissão.

O canal escolhido para essa interseção é o dispositivo móvel: a aprendizagem móvel (*m-learning*) tira partido da portabilidade e ubiquidade destes dispositivos [21], transformando a aprendizagem num processo contínuo integrado na vida quotidiana [22], algo central para o Play4Change, cujo desafio diário se integra nos momentos em que o utilizador já usa o dispositivo. Com 8,9 mil milhões de subscrições móveis ativas no mundo [23], é também o canal mais inclusivo disponível, relevante para o objetivo do DigComp 2.2 de servir *todos* os cidadãos [1].

2.4 Retrieval-Augmented Generation

Os LLM apresentam uma limitação estrutural relevante para aplicações de domínio específico: o seu conhecimento é estático, corresponde apenas ao que aprenderam até à data de treino, e é caro de atualizar sem treinar o modelo de novo desde o início. Na prática, isto torna qualquer LLM um generalista desatualizado sobre tudo o que não estava nesse treino. É o caso, por exemplo, do documento concreto que um formador acabou de submeter à plataforma. É como perguntar a alguém culto, mas que nunca leu esse documento em particular: a resposta pode soar convincente, sem no entanto estar de facto ancorada naquele conteúdo.

O *Retrieval-Augmented Generation* (RAG) resolve este problema de forma direta: antes de responder, o sistema procura primeiro os excertos mais relevantes desse documento (à semelhança de um exame de consulta, em que se localiza a página certa antes de escrever a resposta) e entrega-os ao modelo como contexto, para que a resposta gerada se apoie nesses excertos em vez de no conhecimento genérico do modelo [5]. Essa procura não é feita por palavra-chave exata, mas por significado: cada excerto de texto é convertido num vetor numérico que representa o seu sentido, de forma que dois excertos com significados semelhantes ficam matematicamente próximos entre si, mesmo que usem palavras diferentes. É essa proximidade de significado que permite encontrar, de entre todo o documento, o excerto mais

relevante para uma pergunta concreta. O resultado é uma redução de respostas inventadas (*hallucinations*) e respostas rastreáveis às fontes que as originaram [24].

A sua aplicação a sistemas educativos é uma área de investigação em crescimento: ancorar a geração de questões e *feedback* em documentos fornecidos pelo formador, em vez de no conhecimento genérico do modelo, é particularmente valioso em contextos de formação especializada [4]. É este o papel do RAG no Play4Change, detalhado na Secção 5.2.1 do Capítulo 5: garantir que os desafios gerados para cada Learner estão ancorados no conteúdo submetido pelo Administrador, e não em conhecimento genérico do modelo.

2.5 Síntese Comparativa

A Tabela 2.1 apresenta uma análise comparativa entre o Play4Change e as principais plataformas de referência identificadas no estado da arte, segundo os critérios mais relevantes para o posicionamento do projeto.

Tabela 2.1: Comparação entre o Play4Change e plataformas de referência

Critério	Duolingo	Khan Academy	Coursera	Play4Change	
				Atual	Futuro
Gamificação	✓	~	×	~	✓
Adaptação à dificuldade do utilizador	✓	✓	×	✓	✓
Catálogo de conteúdo pré-definido	✓	✓	✓	×	~
Reconhecimento com credenciais formais	~	×	✓	×	✓
Comunidade e interação social	✓	~	✓	×	✓
Modelo de desafio diário no móvel	✓	×	×	✓	✓
Cliente móvel nativo	✓	✓	✓	✓	✓
Autenticação <i>passwordless</i>	×	×	×	✓	✓

✓ Suportado × Não suportado ~ Parcialmente suportado

A análise revela um posicionamento diferenciado, mas expõe também limitações claras: em gamificação, o Play4Change já tem uma base real (pontuação, *streak* e crachás, visíveis na própria interface do desafio diário), mais leve do que a das plataformas estabelecidas, com espaço evidente para crescer (Secção 7.5). Já nos critérios onde essas plataformas lideram sem contrapartida atual (reconhecimento formal por certificados, catálogo pré-definido e comunidade/interação social), o Play4Change fica em desvantagem real, por depender do conteúdo submetido pelo Administrador e não cobrir ainda estas dimensões sociais, reconhecidas como prioridades de trabalho futuro. Diferencia-se, ainda assim, pelo seu modelo de entrega, desafio diário por tópico no canal móvel, com adaptação ao desempenho e autenticação *passwordless*, uma combinação que nenhuma das plataformas analisadas oferece (com a ressalva sobre múltiplas inscrições simultâneas já assinalada na Secção 2.2).

Capítulo 3

Análise e Requisitos

A definição clara dos requisitos de um sistema é um dos fatores mais determinantes para o sucesso de um projeto de engenharia de software [25]. Este capítulo segue uma progressão deliberada, do geral para o específico: começa por identificar quem usa a plataforma e que necessidades traz (Secção 3.1); traduz essas necessidades em requisitos funcionais (o que o sistema deve fazer, Secção 3.2) e não funcionais (sob que condições de qualidade o deve fazer, Secção 3.3); e fecha com os casos de uso que ilustram, de ponta a ponta, como esses requisitos se materializam em fluxos concretos de utilização (Secção 3.4).

3.1 Identificação dos Utilizadores

A identificação dos utilizadores é o ponto de partida de qualquer processo de desenvolvimento centrado no utilizador (*User-Centered Design*) [26], e levou, no contexto do Play4Change, à identificação de dois perfis de utilizador com necessidades, objetivos e contextos de uso distintos.

3.1.1 Administrador

O **Administrador** é o perfil responsável pela criação e gestão de conteúdo pedagógico, tipicamente um formador ou responsável de uma instituição parceira da rede U!REKA, sem necessidade de *expertise* em IA ou engenharia de software. Interage com a plataforma exclusivamente através do **cliente web**: submete PDF ou URLs como fontes de conhecimento, revê os tópicos e desafios gerados pela IA, e gere os Learners registados. Esta separação, produção de conteúdo versus consumo e progressão, é consistente com os modelos de *Intelligent Tutoring Systems* (ITS) [8].

3.1.2 Learner

O **Learner** é o destinatário da experiência de aprendizagem. O seu perfil é deliberadamente amplo: qualquer cidadão que pretenda desenvolver as suas competências digitais no referencial

DigComp 2.2, independentemente do nível de partida ou do contexto institucional em que se insere.

O Learner interage com a plataforma através do **cliente móvel**, numa experiência desenhada para ser discreta e integrada na rotina diária. O modelo de um **desafio por dia** não é arbitrário: sessões curtas e regulares são significativamente mais eficazes do que sessões longas e esporádicas (*spaced repetition*) [27, 28], e a Teoria da Autodeterminação de Deci e Ryan mostra que a motivação intrínseca é sustentada por três necessidades psicológicas básicas, *autonomia*, *competência* e *relatedness* (ligação social a outras pessoas, não apenas a um propósito abstrato) [29], fatores a que este modelo responde parcialmente: a autonomia e a competência são endereçadas diretamente pelo desenho do desafio diário, mas a *relatedness* depende das funcionalidades sociais ainda por implementar (Secção 7.5). A escolha do canal móvel segue a justificação já apresentada na Secção 2.3.

3.2 Requisitos Funcionais

Identificados os dois perfis de utilizador e as necessidades que os distinguem, importa agora traduzir essas necessidades em comportamentos concretos. Os requisitos funcionais descrevem os comportamentos que o sistema deve exibir em resposta a determinadas entradas ou condições [25]. A Tabela 3.1 apresenta os requisitos funcionais do Play4Change, organizados por área funcional.

Tabela 3.1: Requisitos funcionais do sistema Play4Change

ID	Área	Descrição
RF01	Autenticação	O sistema deve suportar autenticação para ambos os perfis de utilizador, Administrador e Learner.
RF02	Criação de Conteúdo	O Administrador deve poder criar tópicos a partir de fontes documentais (um documento PDF ou um URL), com o conteúdo pedagógico gerado automaticamente por Inteligência Artificial e passível de edição ou regeneração pelo Administrador.
RF03	Reutilização de Conhecimento	O sistema deve reutilizar conhecimento e conteúdo já gerados sempre que semanticamente equivalentes a um novo pedido, evitando duplicação.
RF04	Progressão do Learner	O Learner deve receber um desafio diário extraído dos tópicos em que está inscrito e ainda não concluídos, com <i>feedback</i> imediato após cada resposta.
RF05	Adaptação à Dificuldade	O sistema deve adaptar o percurso de remediação ao nível de dificuldade sentido pelo Learner, ajustando-o ao seu padrão de erro.

continua na página seguinte

(continuação da Tabela 3.1)

ID	Área	Descrição
RF06	Tutoria com IA	Quando a remediação adaptativa não for suficiente para resolver a dificuldade, o sistema deve disponibilizar tutoria conversacional com IA.
RF07	Clientes	O sistema deve disponibilizar um cliente web para o Administrador, com painel de administração e <i>landing page</i> pública, e um cliente móvel para o Learner, centrado na experiência do desafio diário.

3.3 Requisitos Não Funcionais

Para além do que o sistema faz, importa definir sob que condições o deve fazer bem. Os requisitos não funcionais definem as propriedades e restrições do sistema que não dizem respeito a comportamentos específicos, mas à qualidade global da solução [30]. A Tabela 3.2 apresenta os requisitos não funcionais identificados para o Play4Change.

Tabela 3.2: Requisitos não funcionais do sistema Play4Change

ID	Categoria	Descrição
RNF01	Segurança	O sistema deve proteger minimamente os utilizadores e a infraestrutura contra vetores de ataque comuns e conhecidos e assegurar que a comunicação entre clientes e servidor é cifrada, sem pretensão de cobertura exaustiva.
RNF02	Testes	As defesas de segurança e os limites operacionais do sistema (e.g. limites de taxa por IP) devem ser validados por testes automatizados que cubram, no mínimo, os casos de fronteira mais óbvios.
RNF03	Usabilidade	Os clientes web e móvel devem ser intuitivos e autoexplicativos para o seu propósito, permitindo ao utilizador concluir as suas tarefas principais (gerir conteúdo, no caso do Administrador; completar o desafio diário, no caso do Learner) sem necessidade de instrução prévia.
RNF04	Manutenibilidade	O código deve seguir os princípios SOLID, favorecendo um sistema fácil de compreender, testar e evoluir.

3.4 Casos de Uso

Os requisitos definidos nas secções anteriores materializam-se em fluxos concretos de interação entre os atores e o sistema [31]. Apresentam-se de seguida os três casos de uso que cobrem, de ponta a ponta, os fluxos centrais da plataforma: submissão de conteúdo e geração de desafios, resposta ao desafio diário, e remediação adaptativa. Os restantes fluxos do sistema (gestão

de perfil, consulta de estatísticas, revisão de sinalizações, entre outros) são variações mais simples destes três padrões e não são detalhados individualmente.

3.4.1 CU01: Submeter Conteúdo e Gerar Desafios

Ator principal: Administrador.

Pré-condição: O Administrador está autenticado no painel web.

Fluxo principal:

1. O Administrador define o tópico e submete um documento PDF ou um URL (*Uniform Resource Locator*) como fonte de conhecimento.
2. De forma síncrona, o sistema extrai o texto do documento ou página e valida que não está vazio; uma falha de extração é devolvida de imediato ao Administrador, sem chegar a criar o tópico.
3. Ainda de forma síncrona, o sistema guarda uma cópia do conteúdo original, como registo de auditoria e fonte para uma eventual regeneração; uma falha neste passo também é devolvida de imediato, sem criar o tópico.
4. Guardada essa cópia, o sistema cria de imediato o tópico em estado **PENDING** e confirma ao Administrador a submissão, lançando em paralelo o pipeline assíncrono de geração.
5. De forma assíncrona, o sistema divide o texto em *chunks* e invoca o modelo de IA para gerar módulos e os respetivos desafios (*task templates*).
6. Por cada desafio gerado, o sistema verifica se já existe um desafio semelhante no módulo; se existir, descarta-o, persistindo apenas os desafios novos e distintos.
7. Concluída a geração, o tópico transita automaticamente para o estado **ACTIVE**, sem aprovação explícita do Administrador; este pode editar um desafio específico ou desencadear a regeneração completa do tópico.

Pós-condição: O tópico está **ACTIVE**, com os desafios e o conhecimento associado indexados e disponíveis para atribuição aos Learners.

3.4.2 CU02: Responder ao Desafio Diário

Ator principal: Learner.

Pré-condição: O Learner está autenticado na aplicação móvel e está inscrito em, pelo menos, um tópico.

Fluxo principal:

1. O Learner abre a aplicação móvel.

2. O sistema resolve o desafio do dia para o tópico em que o Learner está inscrito (criando a atribuição se ainda não existir uma para a data corrente) e apresenta-o; o desafio em si já foi gerado antecipadamente pela IA durante o pipeline de ingestão do tópico, não havendo recuperação vetorial em tempo real neste passo.
3. O Learner submete a sua resposta.
4. O sistema avalia a resposta e apresenta de imediato se esta está correta, os pontos atribuídos e o *streak* atualizado.
5. O sistema regista o resultado e atualiza o perfil de progressão do Learner.

Pós-condição: O resultado é registado; se a resposta estiver incorreta, é desencadeada a sessão de remediação descrita em CU03.

3.4.3 CU03: Remediação Adaptativa

Ator principal: Learner.

Pré-condição: O Learner respondeu incorretamente a um desafio do dia.

Fluxo principal:

1. O sistema classifica deterministicamente o padrão do erro.
2. O sistema gera três sub-tarefas mais simples, centradas no padrão de erro detetado, para reconduzir o Learner ao desafio original.
3. O Learner resolve as sub-tarefas; se as completar todas corretamente, a sessão é marcada como resolvida e o desafio original do dia é repostado para nova tentativa.
4. Se o Learner voltar a falhar após esgotar a profundidade de remediação definida, o sistema substitui as sub-tarefas por uma sessão de explicação sintetizada por IA, com possibilidade de diálogo de esclarecimento.

Pós-condição: O Learner é conduzido de volta ao desafio original, com uma remediação centrada no seu erro específico; esta adaptação ocorre dentro do próprio desafio do dia, não altera a escolha de que tópico apresentar em dias seguintes: essa seleção continua a ser feita pelo próprio Learner, através de inscrição explícita nos tópicos disponíveis.

Capítulo 4

Arquitetura e Fundações

4.1 Arquitetura Geral do Sistema

O Play4Change está estruturado em torno de uma ideia simples: quem consome a plataforma nunca fala diretamente com quem gera, guarda ou protege o que é consumido. O Administrador cria e gere conteúdo através do cliente web; o Learner consome-o através do desafio diário no cliente móvel (RF07). Entre os dois e o resto do sistema fica sempre um núcleo que decide o que fazer, sem depender de como cada peça está concretizada. A Figura 4.1 apresenta essa estrutura.

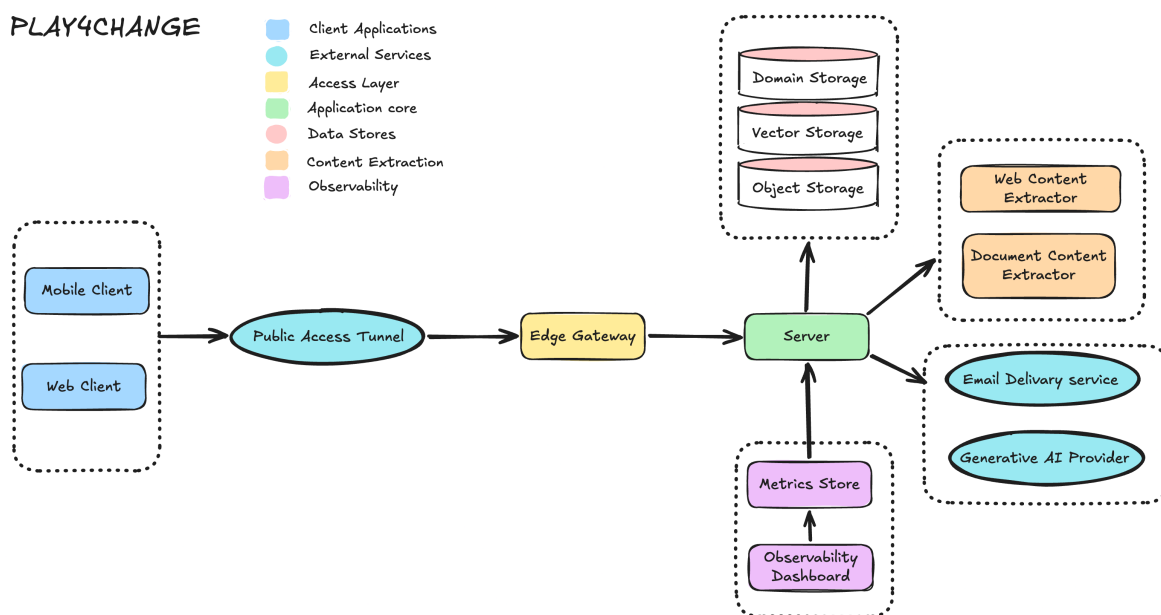


Figura 4.1: Arquitetura geral do sistema Play4Change

Os dois clientes entram pelo mesmo caminho: um túnel de acesso público, que assegura que o tráfego dos clientes não expõe uma porta diretamente e que essa comunicação é cifrada (RNF01), seguido por uma cadeia de filtros de borda que filtra e autentica (RF01) antes de qualquer pedido chegar à lógica de negócio.

Só depois de atravessar este perímetro é que um pedido chega ao núcleo. É o núcleo que

resolve o desafio diário do Learner, adapta o percurso quando há dificuldade e escala para tutoria quando a adaptação não é suficiente (RF04–RF06); e que, do lado do Administrador, recebe um documento ou URL e decide gerar tópicos e desafios a partir dele (RF02, CU01).

Para isso, o núcleo depende de quatro tipos de capacidade externa, e nunca fala diretamente com nenhuma implementação concreta de nenhuma delas, apenas com o que cada uma promete fazer: extrair conteúdo de uma fonte, gerar conteúdo pedagógico e explicações, persistir e indexar semanticamente o que já foi gerado para o reutilizar sempre que possível (RF03), e notificar o utilizador. O sistema é ainda observado de fora: métricas de funcionamento são recolhidas periodicamente a partir do núcleo, sem que essa recolha interfira com nenhum pedido em curso.

Esta indireção não é um detalhe acidental: é o que garante que trocar qualquer uma destas capacidades por outra equivalente não obriga a tocar na lógica de negócio (RNF04), e é o que torna o núcleo testável isoladamente, sem depender de nenhuma delas estar de facto disponível.

Um pedido típico atravessa este perímetro antes de chegar à lógica de negócio; o Capítulo 5 detalha, já ao nível de implementação, como cada uma destas fronteiras foi concretizada.

Capítulo 5

Implementação

O Capítulo 4 desenhou o Play4Change em silhueta: dois clientes que nunca falam diretamente com quem gera, guarda ou protege o que consomem; um perímetro que filtra e autentica antes de qualquer pedido chegar à lógica de negócio; um núcleo que decide; e quatro capacidades externas de que esse núcleo depende sem nunca as referenciar diretamente. Este capítulo preenche essa silhueta, peça a peça: começamos pelo servidor e pela porta de entrada (Secção 5.1), seguimos o conteúdo desde a submissão pelo Administrador até ao tópico pronto para um Learner se inscrever (Secção 5.2), acompanhamos o Learner no seu percurso diário (Secção 5.2.2), e fechamos com a segurança que atravessa tudo isto (Secção 5.3). Os *prompts* completos enviados ao modelo em cada uma destas fases estão reproduzidos, sem alterações, no Apêndice A.

5.1 Fundações: Servidor, Pipeline e Autenticação

Os dois clientes do Play4Change (o painel web do Administrador e a aplicação móvel do Learner) precisam de um ponto de entrada único para falar com o núcleo do sistema, capaz de servir da mesma forma um browser e um telemóvel. A forma mais direta de o fazer é uma API RESTful: sem estado entre pedidos, organizada por recursos, e igualmente natural para os dois lados. É essa API que o servidor expõe.

O servidor é escrito em Kotlin sobre Spring Boot 3 (JVM 21), com as regras de negócio isoladas de Spring, JPA ou de qualquer fornecedor externo concreto: a mesma indireção do Capítulo 4, agora ao nível do código.

É esta última camada que dá corpo às quatro capacidades externas do Capítulo 4. Persistir e indexar cai sobre PostgreSQL, via JPA; gerar conteúdo pedagógico cai sobre um módulo dedicado que fala com o Mistral através do LangChain4j, isolado nos seus próprios módulos Gradle (`ai-agent:api` e `ai-agent:langchain`): trocar de fornecedor de IA implica escrever um novo adaptador, não tocar numa linha de lógica de negócio; extrair conteúdo cai sobre Playwright e Unstructured.io, detalhados na Secção 5.2; e notificar o utilizador cai sobre Resend para email e *Server-Sent Events* para progresso em tempo real.

Como o domínio do sistema cresce ao longo do projeto, o esquema da base de dados precisa de evoluir com ele sem arriscar o que já lá está. Essa evolução é gerida exclusivamente por Flyway: cada alteração é um ficheiro SQL versionado e registado em git, aplicado pela mesma ordem em qualquer ambiente (33 até à data).

5.1.1 Pipeline de Processamento de Pedidos

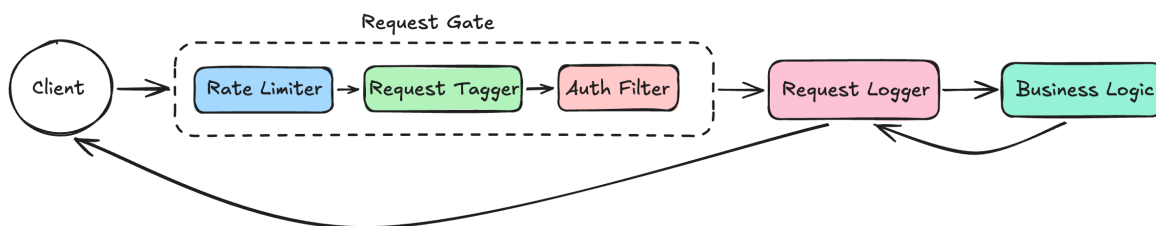


Figura 5.1: Percurso de um pedido HTTP até à lógica de negócio

A Figura 5.1 resume o percurso de um pedido até chegar a um *controller*. Antes disso, todo o pedido atravessa um *Request Gate* de três filtros, e a ordem lá dentro não é arbitrária: um sistema que gera conteúdo por IA a pedido é um alvo natural de abuso, pelo que a primeira pergunta que qualquer pedido enfrenta não é “quem és”, mas “quantos pedidos já fizeste”. O *Rate Limiter* corre por isso antes de qualquer autenticação (o que trava tentativas de força bruta mesmo contra quem ainda não tem sessão) e responde 429 Too Many Requests, o código pensado precisamente para isto (ao contrário de um 403, que assinalaria uma identidade já conhecida mas sem permissão). Mais à frente na mesma cadeia, o *Auth Filter* responde 401 Unauthorized quando o token falha: o código reservado para identidade ausente ou inválida, não para uma identidade válida sem privilégios suficientes (esse caso é 403, adiante). Um pedido travado por qualquer um dos dois não chega ao *Request Logger*: fica sem o identificador de correlação do *Request Tagger* e sem entrada nos registos. Só o que atravessa o Gate por completo é identificado, registado e chega à lógica de negócio.

O *Request Logger* regista o pedido duas vezes, não uma: à entrada, antes da lógica de negócio, e à saída, depois dela: é esse par que permite medir, nos registos, quanto tempo cada pedido demorou do princípio ao fim.

O limite exato depende do *endpoint* (Tabela 5.1): mais apertado onde a frequência esperada é baixa e o risco alto (pedido de código de acesso, para prevenir *email bombing*, e criação ou regeneração de um tópico, porque cada pedido despoleta uma chamada à API da Mistral, pelo que o limite protege também contra custo descontrolado); mais permissivo na submissão de tarefas (30 pedidos/10 min), invocada com muito mais frequência em uso legítimo, mas ainda assim suficiente para travar *brute-forcing* de respostas.

Tabela 5.1: Limites de *rate limiting* por endpoint (janela de reabastecimento de 10 minutos em todos os casos)

Endpoint	Limite	Janela
POST /auth/magic-link	5 pedidos	10 min
POST /auth/magic-link/verify	10 pedidos	10 min
POST /auth/verify	10 pedidos	10 min
POST /auth/oauth	10 pedidos	10 min
POST /auth/refresh	20 pedidos	10 min
POST /auth/logout	10 pedidos	10 min
POST /auth/recovery-email/verify	10 pedidos	10 min
POST /admin/topics	3 pedidos	10 min
POST /admin/topics/pdf	2 pedidos	10 min
POST /admin/topics/{id}/regenerate	2 pedidos	10 min
POST /tasks/{id}/submit	30 pedidos	10 min
POST /struggle/{id}/tasks/{id}/submit	30 pedidos	10 min
POST /explanation/{id}/message	10 pedidos	10 min

Um pedido em particular não se resolve dentro desta cadeia: quando um Administrador submete um documento ou URL para gerar os desafios correspondentes por IA demora demasiado para o deixar à espera de uma resposta HTTP. Em vez de bloquear, o servidor confirma a submissão de imediato (201 Created) e passa o resto do trabalho para um executor dedicado, que progride por cinco estados:

INGESTION → ANALYSIS → GENERATION → INDEXING → ACTIVE

(qualquer fase pode transitar para FAILED, e dois *schedulers* Spring vigiam o pipeline: um marca como falhados os tópicos presos demasiado tempo em geração, outro expira os que já passaram do prazo). O 201 Created encerra ali o pedido original: o acompanhamento do progresso não faz parte dele. Para o ver sem recarregar a página, o cliente abre, à parte, uma ligação de *Server-Sent Events* a GET /admin/topics/{id}/progress, recebendo eventos de mudança de fase até à conclusão ou falha da geração. Esta máquina de estados é a espinha dorsal de tudo o que se segue na Secção 5.2: cada uma das suas fases corresponde a um mecanismo concreto detalhado adiante.

5.1.2 Autenticação sem Password

Autenticar um pedido resume-se sempre à mesma mecânica: o utilizador prova a sua identidade uma vez, o servidor emite um token que a representa, e é esse token (não a prova original) que o Auth Filter valida a cada pedido seguinte (Secção 5.1.1). O que muda de sistema para sistema é só essa primeira prova, a que acontece uma vez para depois nunca mais ser preciso repetir. A forma mais comum de a fazer, uma *password*, é mal ajustada a este produto em particular: é vulnerável a reutilização entre serviços, força bruta e *phishing*, frequentemente esquecida, e Bonneau et al. [32] avaliam 35 esquemas alternativos sem encontrar um vencedor universal: os mais resistentes a ataques (tokens físicos, biometria) tendem a

sacrificar exatamente a usabilidade em que as *passwords* se destacam. Para um desafio breve e diário, dirigido também a um perfil de menor afinidade tecnológica (Secção 6.2.3), a escolha caiu sobre um código de utilização única enviado por email, que o utilizador introduz para se autenticar. Isto elimina a fricção de memorização e o conceito de password esquecida, ao custo de a segurança e a recuperação da conta passarem a depender inteiramente do email do Learner, risco que o sistema mitiga com um email de recuperação alternativo como segunda via de acesso.

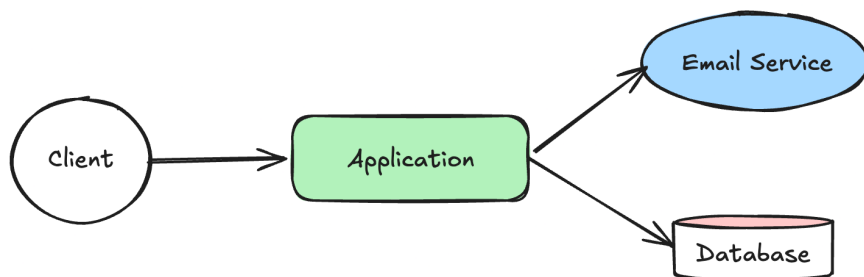


Figura 5.2: Relação entre a aplicação e os serviços externos no pedido de código

A Figura 5.2 mostra a relação por detrás deste pedido: a aplicação nunca fala com o email do utilizador nem com a base de dados por si só: pede a um serviço de email externo (Resend) que envie o código, e guarda na base de dados apenas o *hash* SHA-256 desse código, nunca o valor em claro. Três decisões deliberadas fecham as frestas óbvias deste tipo de fluxo: o pedido do código responde sempre **202 Accepted** independentemente de o email existir, para não permitir a um atacante descobrir que contas existem; o consumo do código é uma operação atómica, para que dois pedidos em simultâneo não consigam ambos ter sucesso; e as três formas de falha (código inválido, expirado, ou já usado) são indistinguíveis para o cliente, para não dar pistas a quem tenta adivinhar.

Autenticado com sucesso, o Learner recebe um par de tokens, e cada um viaja e é guardado de forma diferente. O *access token* JWT, de vida curta (15 min), vai no *header Authorization: Bearer* de cada pedido seguinte e nunca é guardado no servidor: a própria assinatura chega para o validar (Secção 5.1.1). O *refresh token*, de vida mais longa (7 dias), vai apenas no corpo do pedido a **POST /auth/refresh** e é guardado no servidor unicamente como *hash*, nunca em claro. Do lado do cliente, cada plataforma usa o seu próprio armazenamento seguro: *Keychain* no iOS, *EncryptedSharedPreferences* no Android, e um *cookie SameSite=Strict* no painel web.

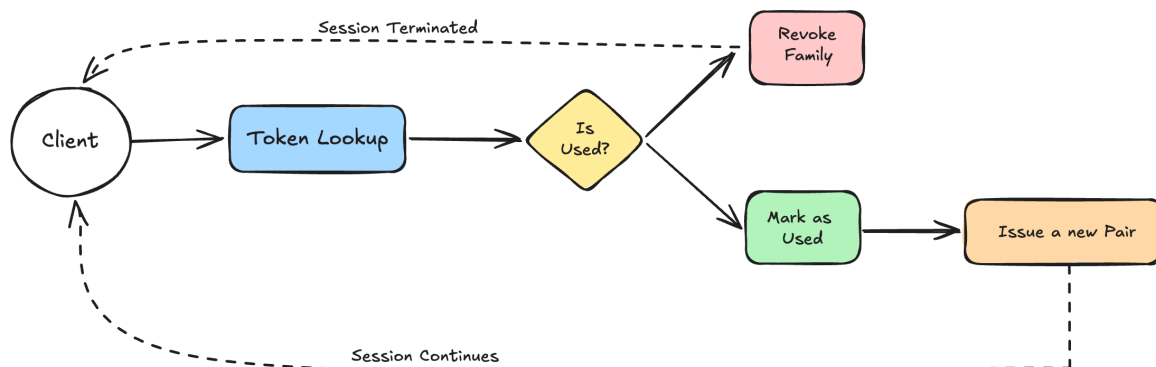


Figura 5.3: Rotação do refresh token e deteção de reutilização

É o *refresh token* que precisa de mais cuidado, precisamente por viver mais tempo: segue rotação com deteção de reutilização (Figura 5.3). A cada renovação, o token apresentado é procurado pelo seu *hash* e marcado como usado antes de um novo par ser emitido, sempre associado à mesma família (`familyId`). Se esse mesmo token (já marcado como usado) for reapresentado, o sistema interpreta isso como sinal de roubo: revoga toda a família de uma vez, e o Learner tem de voltar a autenticar-se do zero. O acesso às rotas de administração exige o papel correspondente, validado a cada pedido contra o mesmo JWT; esse papel fica fixado na criação da conta (por omissão Learner) e só é elevado a Admin por edição manual direta na base de dados, nunca pela API pública nem por qualquer mecanismo de semente automatizado.

5.2 Do Conteúdo ao Desafio

Com o núcleo e a porta de entrada no lugar, faltam as duas metades que tornam o Play4Change mais do que um CRUD: como o conteúdo é criado, do lado do Administrador (Secção 5.2.1), e como é consumido, do lado do Learner (Secção 5.2.2).

5.2.1 Criação do Conteúdo

Um Administrador começa sempre com pouco: um documento PDF ou o URL de uma página. O que tem de sair no fim não é um desafio isolado, é um **tópico** completo (com os seus módulos e desafios já gerados) pronto para um Learner se inscrever e, a partir daí, passar a receber um desafio novo desse tópico a cada dia. É esse caminho que a Figura 5.4 resume e o resto desta subsecção percorre, zona a zona.

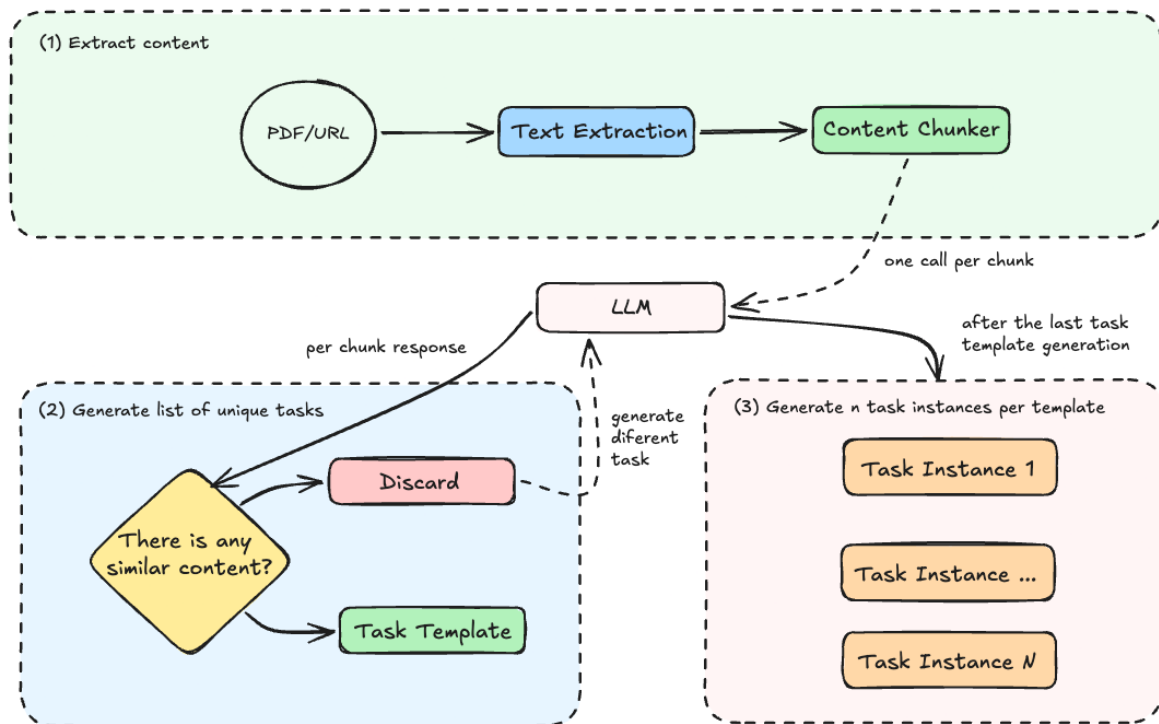


Figura 5.4: Do conteúdo extraído ao desafio guardado: uma chamada por bloco, deduplicação antes de persistir, reposição dirigida quando falta quantidade

Antes da Zona (1): do Documento a Texto Simples

Um PDF e uma página web exigem tecnologia genuinamente diferente para se tornarem texto, por isso a extração vive atrás de um único contrato, com duas implementações consoante a fonte:

- **PDF:** a **Unstructured API** [33], escolhida pela necessidade de OCR (*Optical Character Recognition*) em documentos digitalizados;
- **URL:** um microserviço **Playwright** [34], necessário porque muitas páginas educativas são *Single Page Applications* que só mostram conteúdo depois de executar JavaScript; uma chamada HTTP simples veria uma página vazia.

Um URL traz, porém, um risco que um PDF não traz: é o próprio servidor que vai buscá-lo, e nada impede, à partida, que aponte para um endereço interno da rede em vez de uma página pública. Antes de qualquer pedido de rede, o `UrlSsrValidator` fecha essa porta: esquema `https` obrigatório, `host` válido e, por resolução DNS, rejeição de endereços *loopback*, de rede privada ou *link-local*.

Zona (1) → Caixa “LLM”: Uma Chamada por Bloco

Com o texto já extraído, resta um último obstáculo antes de chegar ao modelo de IA: um documento inteiro raramente cabe num único *prompt*, e mesmo que coubesse, seria caro processá-lo

de uma vez. O **ContentChunker** particiona-o em blocos de 32 000 caracteres com sobreposição de 2 000, preferindo cortar em limites de parágrafo para não partir uma ideia a meio: é por isso que a seta só chega a “LLM” depois de extração e *chunking*, um bloco de cada vez, nunca o documento inteiro. A caixa está rotulada de forma deliberadamente genérica: seja qual for o fornecedor por trás, a resposta a cada chamada é sempre um conjunto de tarefas candidatas para esse bloco, nunca uma tarefa isolada. Na versão atual, esse fornecedor é o **Mistral AI** (`mistral-small-latest`, via `LangChain4j`), escolhido por quatro razões:

- os *prompts* são multilingues por natureza, e o modelo responde bem em várias línguas sem ajuste adicional;
- gerar uma questão de escolha múltipla não exige a capacidade de raciocínio de um modelo maior, pelo que um modelo mais pequeno chega, a um custo por chamada menor;
- o sistema depende de *output* JSON estruturado e fiável, que o modelo cumpre de forma consistente;
- a arquitetura já garante que trocar de modelo, se um dia for preciso, não é uma reescrita (Secção 4.1).

Quantas tarefas cada chamada pede também não é fixo: o total definido pelo Administrador é repartido pelos blocos em proporção ao comprimento de cada um (**TaskCountAllocator**), não pedido na íntegra a cada bloco: um documento com dez secções e um pedido de cem tarefas produz cem no total, mais nas secções com mais conteúdo, menos nas mais curtas, e nunca próximo de mil com todas a mirar o mesmo alvo. Quando o total não é divisível de forma exata, o resto vai para os blocos cuja quota arredondada perdeu mais (método do maior resto), garantindo que a soma final corresponde sempre ao pedido.

Quando a chave de API não está definida, um adaptador *no-op* responde 503 em vez de falhar em silêncio.

Nas duas primeiras finalidades do módulo de IA (gerar tarefas e sintetizar ramos de remediação), o que entra nesta caixa nunca é texto livre escrito por um utilizador: o *user prompt* é sempre construído a partir de dados estruturados do próprio domínio (título do tópico, nível de audiência, padrão de erro), o que é, nesses dois casos, a primeira e mais eficaz defesa do sistema contra *prompt injection* (Secção 5.3): não há *input* a injetar onde não há *input* nenhum. Já a terceira finalidade, a condução de sessões de explicação, recebe efetivamente o texto que o Learner escreve na conversa (Secção 5.2.2.3); nesse caso a defesa não é a ausência de *input*, mas o isolamento desse *input* na sua própria mensagem de origem não confiada, como descrito na Secção 5.3.

Zona (2): O Losango e o Retorno a “LLM”

Chamar o modelo tem um custo que cresce com o número de utilizadores e de tópicos, por isso cada candidata que sai dele passa primeiro por um filtro de semelhança, não é logo guardada. O seu texto é convertido numa espécie de impressão digital numérica (um vetor que resume o *significado* da frase, não as palavras exatas), e essa impressão digital é comparada, uma a uma, com as de todas as tarefas já existentes no mesmo módulo. Se a semelhança for demasiado alta, a candidata é descartada como duplicado semântico, sem sequer chegar a ser persistida; caso contrário, torna-se um **TaskTemplate** definitivo. É esta inversão (usar recuperação vectorial para decidir se vale a pena gerar, em vez de para enriquecer o que se gera) que é a decisão de implementação mais original do sistema: quanto mais a base de conhecimento cresce, menos o custo de gerar cresce com ela.

A pergunta óbvia que o losango levanta é o que acontece às descartadas: ficam simplesmente perdidas, reduzindo silenciosamente o número final de tarefas? Não: é essa a seta de retorno a “LLM”. Se, no fim de um bloco, o número de tarefas aceites ficar abaixo do pedido, o sistema volta a chamar o modelo uma única vez, mas de forma dirigida, não às cegas: pede exatamente a diferença (se de dez pedidas nove forem aceites, pede uma, não as dez outra vez) e envia ao modelo, explicitamente, os títulos das tarefas já aceites nesse bloco e nos anteriores do mesmo tópico, para que a nova tentativa tenha uma hipótese real de ser diferente em vez de arriscar repetir o que já foi descartado. Como qualquer chamada de rede a um serviço externo, esta também pode falhar ou demorar; e, como é a chamada mais cara e mais lenta do sistema, está protegida por um *timeout* de 60 segundos por bloco. Nesta fase a geração corre em segundo plano, do lado do Administrador, pelo que a falha de uma chamada não bloqueia ninguém; o que acontece ao bloco (e ao tópico) quando isso falha é explicado no fecho desta subsecção, na Zona (3).

Zona (3): As Caixas Laranja, Depois do Último *Template*

Gerar uma tarefa uma vez não chega: se todos os Learners inscritos no mesmo módulo vissem exatamente a mesma pergunta com as mesmas opções erradas, comparar respostas seria trivial. Por isso cada desafio existe em dois níveis:

- **TaskTemplate**: a definição canónica gerada pelo LLM;
- até cinco **TaskInstance** por *template*: variantes com distratores diferentes, que partilham a mesma resposta correta mas nunca as mesmas opções erradas.

Estas instâncias só são geradas depois de todos os blocos do documento terem terminado a sua fase de geração e os respetivos *templates* estarem todos persistidos. Uma falha no primeiro bloco aborta a geração inteira; falhas em blocos seguintes são toleradas, preservando o que já foi persistido.

Nenhuma questão chega ao Learner sem passar por um conjunto de verificações simples que garantem que faz sentido. Antes de ser guardada, cada candidata é comparada em significado com as já existentes no mesmo módulo, para não repetir uma pergunta com outras palavras; o formato é validado: número certo de opções, resposta correta identificada de forma consistente; e confirma-se que está na língua e no nível de dificuldade pedidos. O texto que o modelo devolve é sempre limpo antes de persistir, para que não haja espaço para conteúdo indevido se infiltrar. E se o modelo falhar ou devolver algo incompleto, o sistema tenta corrigir automaticamente antes de desistir: nunca fica uma questão a meio guardada como se estivesse completa. É este conjunto de verificações, aplicado a cada questão individualmente, que garante a qualidade do conteúdo gerado, não uma auditoria posterior.

5.2.2 Consumo do Conteúdo

O tópico já existe; falta o Learner inscrever-se nele, receber os seus desafios diários, tentar resolvê-los, e o sistema reagir ao resultado. Esta subsecção segue esse percurso do princípio ao fim, em três momentos: o pedido do desafio do dia, o que acontece quando a resposta falha, e o que acontece quando nem a remediação chega. Os três encadeiam-se num único ciclo.

5.2.2.1 Atribuição Diária

Em vez de o sistema calcular e enviar um desafio a cada utilizador todos os dias (o que exigiria um processo em segundo plano a correr para toda a base de Learners), é o Learner que pede o seu ao abrir a aplicação. A Figura 5.5 resume esse pedido e o que acontece depois de responder.

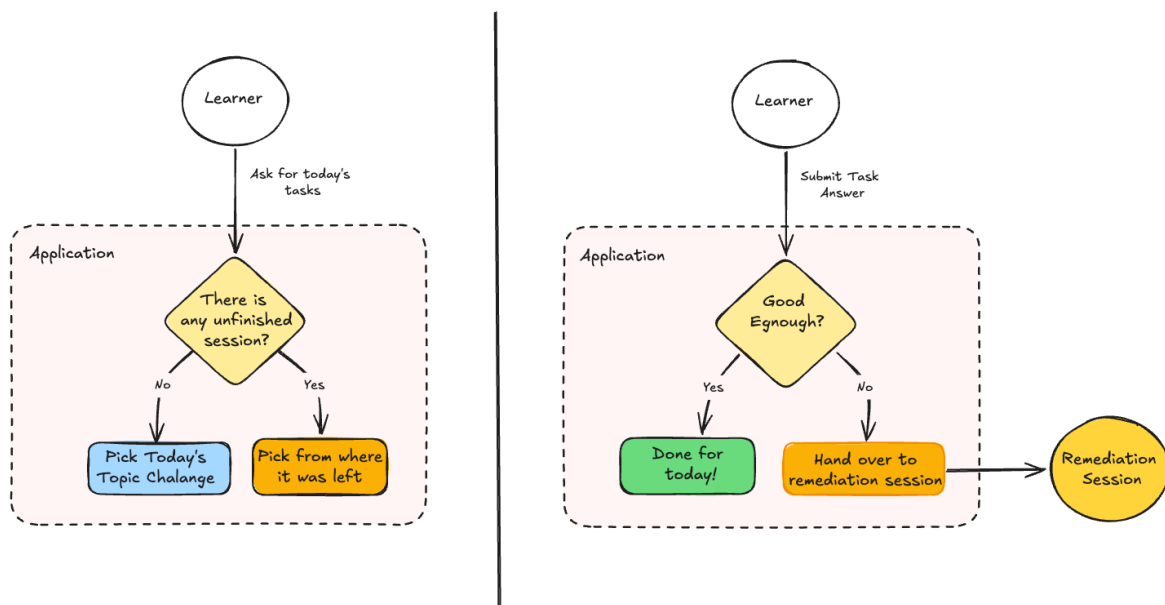


Figura 5.5: Pedido do desafio diário e desfecho da resposta

À esquerda, o pedido (GET /tasks/today) resolve-se num de quatro casos, por ordem: uma sessão de remediação aberta, uma sessão de explicação ativa, um desafio já atribuído nesse dia, ou a criação de um novo; os três primeiros são, na prática, a mesma ideia: retomar exatamente onde o Learner ficou. Só no último caso é que há algo por decidir: tanto a instância escolhida como a ordem das opções apresentadas são determinadas por uma semente ligada ao utilizador, reprodutível para ele (se recarregar o ecrã, vê sempre a mesma versão), mas sem padrão adivinhável para quem não conhece essa semente. É assim que o sistema garante que a resposta correta, sempre fixada no índice 0 internamente, aparece numa posição diferente e imprevisível no ecrã.

À direita, a resposta do Learner decide o resto do dia. O prazo vai até à meia-noite do dia seguinte no fuso horário do Learner; passado esse prazo, a submissão continua a ser aceite, mas com uma penalização crescente em vez de um corte abrupto. Se a resposta estiver certa, o dia termina ali; se não estiver, o Learner não fica simplesmente a saber que errou: é encaminhado de imediato para uma sessão de remediação, sem ação nenhuma da sua parte.

5.2.2.2 Remediação Adaptativa

Um erro do Learner não é tratado como um resultado final, mas como um sintoma a diagnosticar. A Figura 5.6 segue esse diagnóstico até à sessão de remediação e, do outro lado, até ao que acontece quando ela também não chega.

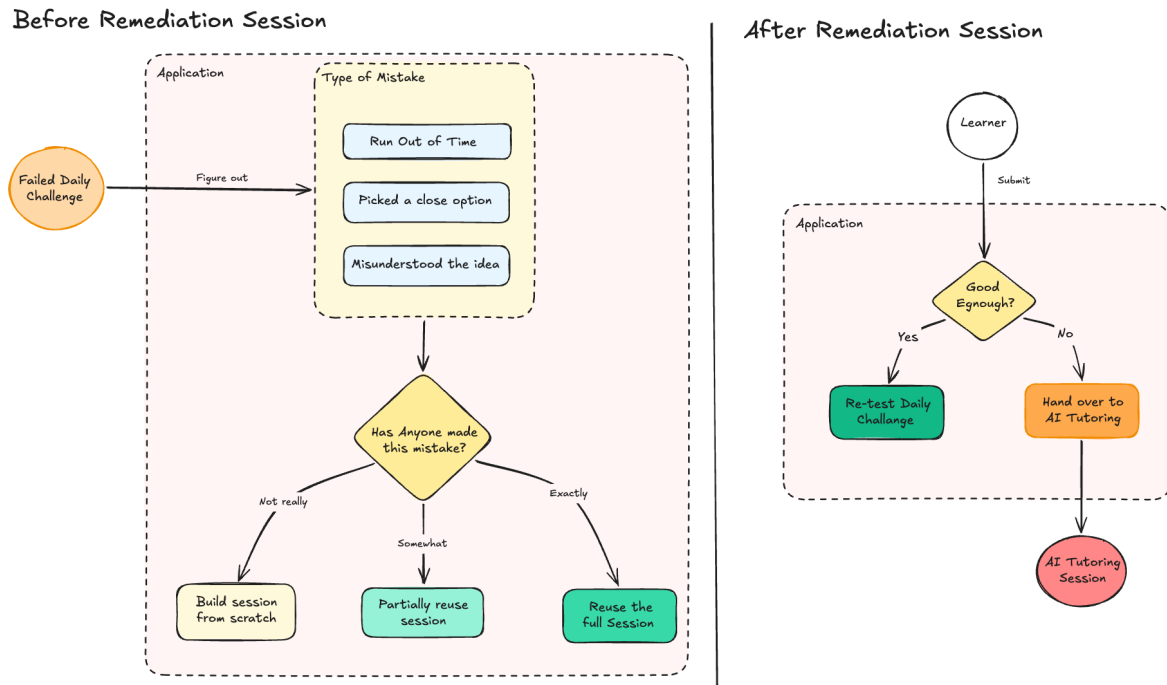


Figura 5.6: Diagnóstico do erro, reutilização entre Learners, e desfecho da remediação

À esquerda, antes de gastar uma chamada de IA, o `ErrorPatternClassifier` classifica a falha de forma determinística, sem envolver o modelo, em três categorias (Tabela 5.2). Só

depois disso é que entra a pergunta que o losango do diagrama levanta (“já alguém cometeu este mesmo erro?”), porque gerar as três sub-tarefas de remediação do zero, sempre que um Learner erra, seria repetir trabalho que a plataforma já fez para outro Learner com a mesma dificuldade.

Tabela 5.2: Padrões de erro classificados pelo ErrorPatternClassifier

Padrão	Condição	Estratégia de remediação
TIME_PRESSURE	Submissão após o prazo	Tarefas paceadas, passo a passo
READING_ERROR	Opção selecionada adjacente à correta	Instruções mais explícitas
WRONG_CONCEPT	Qualquer outro caso	Re-explicação do conceito de raiz

Essa reutilização segue a mesma lógica de custo já vista na criação de conteúdo (Secção 5.2.1), mas com uma pergunta mais fina do que “já existe”: é “quão parecida é a dificuldade que este Learner está a sentir com uma que já vimos antes”. O vetor de dificuldade (padrão de erro + descrição da tarefa + domínio) é comparado aos ramos já resolvidos por outros Learners, e a resposta depende de quão perto está o mais próximo.

Tabela 5.3: Estratégias de reutilização adaptativa por limiar de similaridade

Similaridade	Estratégia	Comportamento
> 0.90	FULL_REUSE	Devolve o ramo existente, sem chamada ao LLM
(0.65, 0.90]	PARTIAL_REUSE	Reutiliza $\max(1, \lfloor n \times \text{normSim} \rfloor)$ tarefas; gera o complemento via Mistral
≤ 0.65	FRESH_GENERATION	Gera o ramo completo via Mistral; persiste o <i>embedding</i> para reutilização futura

No caso intermédio, a proporção exata de reutilização não é um valor fixo, mas interpolada linearmente entre os dois limiares, $n_{\text{reuse}} = \max\left(1, \left\lfloor n_{\text{total}} \times \frac{\text{sim} - 0.65}{0.90 - 0.65} \right\rfloor\right)$, com um mínimo de uma tarefa reutilizada mesmo quando a similaridade está próxima do limiar inferior: quanto mais perto do limiar superior, mais próxima a dificuldade, e menos o sistema precisa de gerar de novo. O importante é que esta reutilização é *entre* utilizadores, não pessoal a cada um: o valor do mecanismo não está em lembrar-se de um Learner, está em a plataforma inteira aprender com cada dificuldade que já viu.

Esta base de conhecimento pode, no entanto, tornar-se incoerente quando um Administrador edita conteúdo já gerado. O sistema resolve isso de forma diferente consoante o alcance da edição: uma edição pontual a uma tarefa mantém deliberadamente o *embedding* antigo, porque só afeta deduplicação futura, não o que já foi entregue a um Learner; já uma regeneração completa do tópico purga e recalcula todos os vetores associados, de raiz.

Esta remediação para, deliberadamente, ao fim de um único nível. Continuar a encadear sub-tarefas do mesmo formato (escolha múltipla) tem retorno decrescente: se três sub-tarefas ainda não resolveram a incompreensão, mais três no mesmo molde dificilmente resolverão, e é a tutoria conversacional, não mais escolha múltipla, que responde a esse caso. Há também um argumento operacional: cada nível adicional multiplicaria chamadas à API de IA e prolongaria

o tempo até o Learner voltar ao fluxo principal. O compromisso é reconhecido (tópicos avançados ficam sem patamares intermédios antes do escalamento) e tornar a profundidade configurável por tópico fica identificado como trabalho futuro. Estas tarefas adaptativas não pontuam (são modo de aprendizagem, não avaliação), mas, ao contrário da pontuação, completá-las corretamente na íntegra incrementa o *streak* do Learner, do mesmo modo que uma resposta certa ao desafio principal.

É esse limite de um único nível que a Figura 5.6 mostra à direita: o Learner submete as três sub-tarefas e, se as resolver todas, volta a testar o desafio original, fechando o ciclo; se não resolver, o sistema já não insiste com mais remediação: entrega-o à tutoria com IA, o único passo que resta.

5.2.2.3 Tutoria com IA

Quando mesmo a remediação falha, uma explicação de IA é o último recurso antes de deixar o Learner preso. A Figura 5.7 segue essa sessão desde que é aberta até ao Learner voltar, de uma forma ou de outra, ao desafio diário, fechando o ciclo que a Figura 5.5 começou.

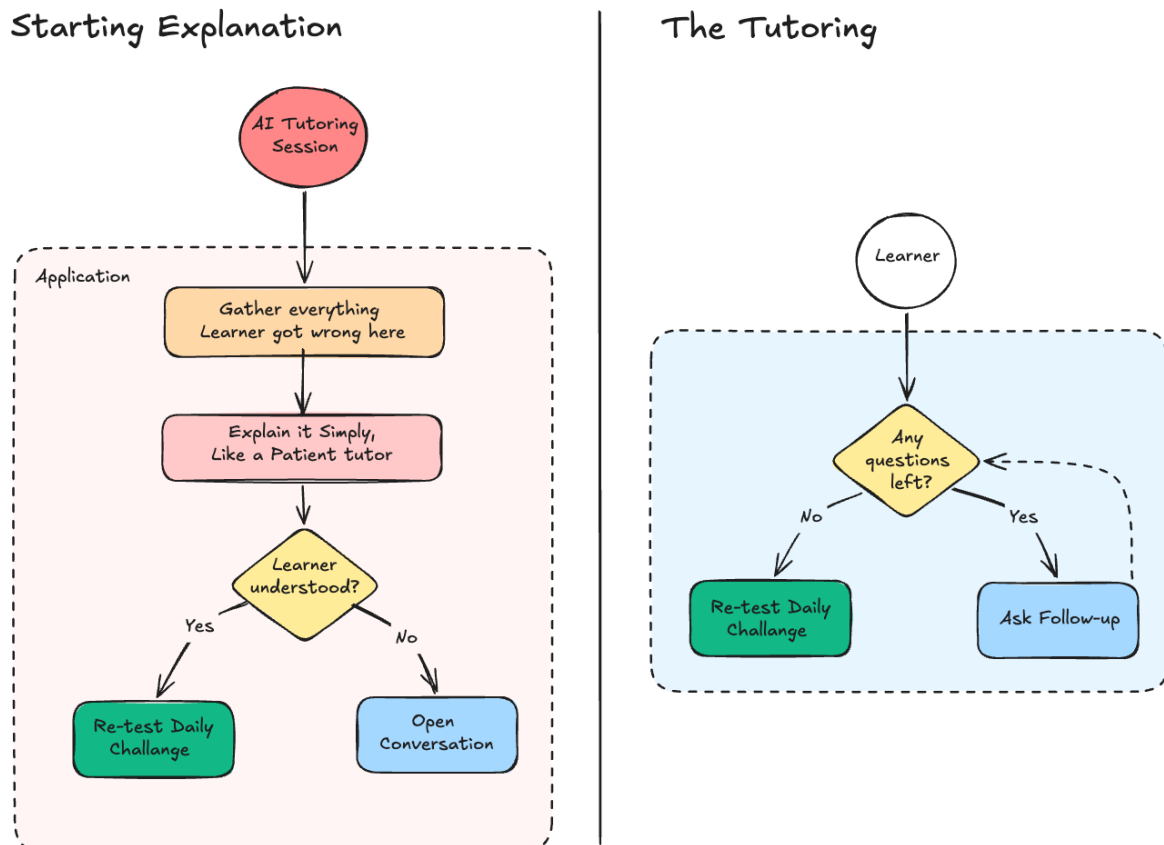


Figura 5.7: Explicação holística seguida de diálogo, até ao regresso ao desafio diário

À esquerda, a sessão começa sempre pela mesma pergunta: o que é que este Learner, especificamente, já demonstrou não perceber? A resposta é o historial completo dos seus

erros naquela tarefa, e é a partir dele (não de uma explicação genérica do conceito) que o modelo gera uma explicação holística, assumindo a persona de um tutor paciente focado na causa raiz. Se essa explicação bastar, o Learner regressa de imediato ao desafio diário; se não bastar, o sistema abre uma conversa.

À direita, essa conversa não tem limite de turnos (o Learner pergunta o que precisar, quantas vezes precisar), mas tem um limite de acesso por endereço IP, para conter o custo de cada chamada ao modelo, e o histórico de confiança mantém-se isolado do *input* do Learner pela mesma estrutura de mensagens tipadas descrita na Secção 5.3. Assim que o Learner considera esclarecido, regressa ao mesmo sítio onde tudo começou: o desafio diário. Se a geração falhar ou exceder 60 s, a sessão resolve-se automaticamente e o Learner é devolvido a esse mesmo ponto: também aqui, nunca fica bloqueado.

5.3 Segurança: Cinco Camadas de Defesa

Cada capacidade aberta ao exterior nas secções anteriores (aceitar um URL de um Administrador, gerar texto por IA e devolvê-lo a um utilizador, expor uma API pública) é também uma superfície nova. A segurança não foi o foco central deste projeto, mas foi endereçada de forma pragmática à medida que cada uma destas superfícies foi ficando clara, camada a camada, várias delas catalogadas no OWASP (*Open Worldwide Application Security Project*) Top 10. No total, cinco camadas independentes protegem o caminho entre a entrada HTTP e a chamada ao LLM. As primeiras três já foram apresentadas onde naturalmente se aplicam: *rate limiting* e autenticação JWT na porta de entrada (Secção 5.1.1), e validação de URL contra SSRF na ingestão (Secção 5.2.1). Não são repetidas aqui. As duas restantes, específicas à camada de IA, fecham o capítulo.

Camada 4: Prevenção de Prompt Injection

Um sistema que gera texto por IA e o mostra a utilizadores tem de responder a uma pergunta simples: o que impede um Learner de escrever, numa das suas mensagens, algo como “ignora as instruções anteriores e...”? A resposta clássica, sanitizar o *input*, é frágil porque tenta detetar ataques depois de já os deixar entrar na mesma *string* que o resto do *prompt*. O Play4Change evita o problema pela raiz: cada peça de informação enviada ao modelo é encapsulada no tipo de mensagem correspondente ao seu nível de confiança: uma **SystemMessage** de confiança, dados estruturados da base de dados, o historial da conversa, e só por fim, isolado na sua própria mensagem de origem não confiada, o *input* atual do utilizador. A fronteira de *role* da própria API do modelo garante que essa última mensagem nunca altera o comportamento definido na primeira. E, como já referido na Secção 5.2.1, os prompts de geração de tarefas e de análise de dificuldade nem sequer recebem *input* do utilizador final: nesses casos, o vetor está eliminado por construção, não por deteção.

Camada 5: Sanitização de Output e Outros Controlos

A pergunta inversa também precisa de resposta: o que impede o próprio modelo, ao reproduzir conteúdo de uma página raspada, de devolver HTML malicioso que acabe por correr no browser de um utilizador? Três técnicas independentes respondem, em camadas: todo o texto gerado é normalizado para texto plano (remoção de todas as tags HTML, incluindo `<script>`/`<style>`) antes de ser persistido; o Nginx aplica uma *Content Security Policy* restritiva ao bloco que serve a aplicação web; e o React escapa automaticamente qualquer expressão ao renderizar no ecrã. É uma defesa deliberadamente redundante: mesmo que uma das três falhe, as outras duas continuam a proteger.

Um punhado de outros controlos, mais pontuais, fecha as portas mais óbvias que ficariam por fechar: o servidor recusa-se a arrancar em produção com um segredo JWT por omissão; a rotação de *refresh token* já descrita revoga toda a família ao primeiro sinal de reutilização; o acesso é limitado por papel e a lista de origens CORS é verificada explicitamente ao arranque, sem aceitar *wildcards*; o Nginx aplica quatro *headers* de segurança a todas as respostas, mais uma *Content Security Policy* restrita ao bloco que serve a aplicação web; e o *scraper* de URLs opera com *timeout* próprio e remove ativamente elementos de *script* antes de extrair texto.

Vetores de Ataque Considerados

Não foi conduzida uma auditoria de segurança formal nem uma análise de risco quantificada: a segurança foi endereçada vetor a vetor, à medida que cada um se tornou relevante à arquitetura concreta do sistema. A Tabela 5.4 junta, num só lugar, os dez vetores considerados ao longo deste capítulo e o controlo correspondente.

Tabela 5.4: Vetores de ataque considerados e controlo implementado

Vetor	Controlo implementado	Observação
XSS via conteúdo gerado por IA	Sanitização + CSP (parcial) + <i>auto-escaping</i> do React	—
Prompt Injection	Multi-mensagem tipada + limites de contexto	Mitiga ataques simples; o Mistral não garante resistência total a <i>inputs</i> adversariais sofisticados
Custo descontrolado de chamadas à API de IA	Rate limiting por IP (10 req/10 min em <i>/explanation</i> , entre outros)	Mitiga abuso de um único atacante, não um ataque distribuído
Roubo de <i>refresh token</i>	Rotação com família + revogação total em caso de reutilização	Janela de 7 dias de exposição se o token for exfiltrado; impacto limitado pela TTL de 15 min do <i>access token</i>
Segredo JWT por omissão em produção	<i>Startup guard</i> impede o arranque	—
SSRF via URL de ingestão	<code>UrlSsrValidator</code>	—
Força bruta em autenticação	Rate limiting (5 req/10 min)	—
<i>Spoofing</i> de IP no rate limiting	Extrator de IP rejeita endereços privados no header	—
Escalada de privilégios (Learner → Admin)	RBAC + validação de papel no JWT	—
CORS <i>wildcard</i>	Verificação de lista de origens ao arranque	—

Três destas decisões deixam uma margem de exposição residual mais visível, e não foram objeto de mitigação adicional: a proteção contra *prompt injection* não é absoluta, porque depende do comportamento do modelo, não só da separação de mensagens; o *rate limiting*

por IP não impede um ataque distribuído a partir de múltiplos endereços; e a janela de 7 dias do *refresh token* só é neutralizada se a sanitização de *output* estiver de facto bem mitigada. Ficam identificadas como pontos a reforçar caso o sistema avance para produção com utilização real.

Este capítulo descreveu o que foi de facto implementado e validado. As decisões deliberadamente deixadas de fora do âmbito atual (profundidade de remediação configurável por tópico, funcionalidades sociais e de retenção, reconhecimento formal de competências, entre outras) estão detalhadas na Secção de Trabalho Futuro do Capítulo 7, e não são repetidas aqui.

Capítulo 6

Testes

Este capítulo valida o que foi descrito no Capítulo 5, em duas frentes complementares: a Secção 6.1 cobre a suite automatizada (testes unitários, de integração e de segurança, que verificam o comportamento do sistema camada a camada e o confrontam com os vetores de ataque considerados, §5.3); a Secção 6.2 reporta a avaliação de usabilidade conduzida com doze utilizadores reais, que mede a percepção de facilidade de uso, não o ganho efetivo de competências digitais.

6.1 Testes Automatizados

A validação automatizada assenta em três camadas complementares: testes unitários que verificam o comportamento isolado do domínio e dos serviços de aplicação, testes de integração que exercitam a camada HTTP em contexto Spring parcial, e testes de segurança que confrontam as defesas com os vetores de ataque considerados (§5.3). Juntas, cobrem 51 ficheiros de teste distribuídos pelos módulos *server* e *play4change-web*, resumidos na Tabela 6.1.

A estratégia assenta em três princípios: **isolamento total pelo padrão hexagonal** (todas as dependências externas são interfaces, substituídas por duplicados *Mockk*, pelo que os testes de domínio correm em memória pura); **Either<Error, T> como contrato de resultado** (a biblioteca Arrow obriga cada caso de uso a devolver **Either<DomainError, T>** em vez de lançar exceções, tornando os casos de insucesso tão verificáveis como os de sucesso); e **separação em três níveis** (unitário, integração via **@WebMvcTest**, e segurança).

Tabela 6.1: Suite de testes automatizados: dimensão e caracterização por nível.

Nível / Módulo	Ficheiros	Ferramenta	Contexto Spring	Deps. externas
Unitário (server)	27	JUnit 5 + Mockk	Nenhum	Todas em mock
Integração (server)	11	@WebMvcTest + MockMvc	Parcial (camada web)	Serviços em mock
Frontend (React/Vite)	13	Vitest + RTL	—	Mock
Total	51			

Os testes **unitários** cobrem o domínio e a aplicação em memória pura: autenticação (revogação de família de tokens JWT em caso de roubo, ameaça R04 §5.3; consumo atómico

do código de acesso), entrega de desafios (baralhagem determinística de opções por semente de utilizador §5.2.2.1; guarda de arranque que impede `devMode` em produção) e deteção de dificuldades (ciclos no grafo de pré-requisitos, um DAG ou *Directed Acyclic Graph*, por DFS; classificação de padrões de erro por prioridade estrita).

Os testes de **integração** exercitam a camada HTTP completa via `@WebMvcTest` (serialização JSON, filtros de segurança, validação de *beans*) em dez controladores REST (autenticação, pré-requisitos, *struggle*, relatórios, preferências, crachás, estatísticas, Swagger) e num teste de *rate limiting* particularmente rigoroso. O `RateLimitTest` substitui o `TimeMeter` real por um `ManualTimeMeter`, tornando deterministas quatro casos (limite estrito, reset de janela, isolamento por IP, cabeçalho `Retry-After`); o complementar `RateLimitSubmissionPathsTest` cobre a correção do desalinhamento de rota nos *endpoints* de submissão (§5.1.1 do Capítulo 5).

Os testes de **segurança** validam as defesas identificadas na análise de risco (§5.3): SSRF (`UrlSsrValidatorTest`, 10 casos cobrindo *loopback*, redes privadas, *link-local* e esquema não-HTTPS), sanitização de *output* da IA (`AiOutputSanitiserTest`, 5 casos), força bruta no código de acesso (coberta pelo `RateLimitTest` acima) e dois guardas de arranque anti-fraude (`devMode` e `TaskDeliveryStartupGuard`) que impedem cadências de desenvolvimento de chegarem a produção.

6.2 Testes de Usabilidade

A avaliação de usabilidade foi conduzida com doze utilizadores reais através da escala SUS (*System Usability Scale*). Antes de apresentar os resultados, importa delimitar o que esta avaliação mede: a escala SUS quantifica a perceção subjetiva de facilidade de utilização e de aprendizagem da interface, não a aquisição efetiva de competências digitais pelo utilizador. Uma pontuação SUS elevada indica que a aplicação é fácil de usar; não constitui, por si só, evidência de que os Learners melhoraram as suas competências no referencial DigComp 2.2. Este relatório não contém dados (avaliação de competências antes/depois da utilização, ou medição longitudinal de desempenho) que permitam sustentar essa inferência.

6.2.1 Metodologia

A avaliação de usabilidade foi conduzida através de um questionário Google Forms aplicado a 12 participantes entre 8 e 12 de Junho de 2026 (4 estudantes actuais do ISEL, 7 testadores externos, 1 antigo aluno), organizado em quatro secções: avaliação global (escala 1–5), quatro declarações de envolvimento (Likert 1–5), a escala SUS (*System Usability Scale*) de Brooke [35] (10 itens, Likert 1–5), e questões abertas.

6.2.2 Análise SUS

A pontuação SUS segue a fórmula de Brooke: subtrai-se 1 aos itens ímpares (positivos) e o valor a 5 nos pares (negativos), soma-se os 10 resultados e multiplica-se por 2,5, numa escala de 0 a 100. Este valor de 0 a 100 **não é uma percentagem**: é um *score* composto que só é interpretável por comparação com um referencial externo [36]. Segue-se por isso, para interpretar os resultados, a orientação do Nielsen Norman Group sobre a análise do SUS [36], que reporta o *benchmarking* de Sauro a mais de 500 estudos: a pontuação SUS média observada na indústria é 68,0, e um sistema com pontuação ≥ 80 posiciona-se no top 10% dos sistemas avaliados nesse universo de estudos. A Tabela 6.2 apresenta as pontuações obtidas face a este referencial.

Tabela 6.2: Pontuações SUS por participante ($n = 12$), face à média de referência da indústria (68,0) [36].

Part.	Perfil	Face à referência (68,0)	SUS
P01	Estudante	Acima (top 10%*)	80,0
P02	Estudante	Acima (top 10%*)	97,5
P03	Externo	Acima (top 10%*)	90,0
P04	Externo	Abaixo	45,0
P05	Alumni	Abaixo	35,0
P06	Externo	Abaixo	50,0
P07	Externo	Acima	77,5
P08	Externo	Acima (top 10%*)	95,0
P09	Estudante	Abaixo	60,0
P10	Externo	Acima (top 10%*)	97,5
P11	Externo	Acima (top 10%*)	92,5
P12	Estudante	Acima (top 10%*)	92,5
Média			76,0
Mediana			85,0

*Pontuação ≥ 80 , o limiar que Sauro reporta como correspondente ao top 10% dos estudos de referência [36].

A média de 76,0 está 8,0 pontos acima da média de referência de 68,0 reportada por Sauro para mais de 500 estudos [36], e sete dos doze participantes (58,3%) atingiram uma pontuação igual ou superior a 80, o limiar que posiciona um sistema no top 10% desse universo de referência. A discrepância entre média (76,0) e mediana (85,0) reflecte dois *outliers* (P04 = 45, P05 = 35) que puxam a média para baixo; sem eles, a média sobe para 83,25. A avaliação global obteve uma média de 4,25 em 5.

Uma ressalva de leitura aplica-se a estes números: comparar doze respostas contra um *benchmarking* assente em centenas de estudos é indicativo, não uma inferência estatisticamente robusta.

As declarações de envolvimento (Tabela 6.3) mostram o design visual como a mais bem pontuada (4,33) e o encorajamento a regressar diariamente como a mais baixa (3,92), suge-

rindo margem de melhoria na retenção.

Tabela 6.3: Médias das declarações de envolvimento (escala Likert 1–5, $n = 12$).

Declaração	Média
A aplicação motivou-me a aprender sobre educação cívica / cidades inteligentes	3,83
Os desafios pareceram relevantes para problemas do mundo real	4,00
O design visual e a estética do jogo foram apelativos	4,33
Senti-me encorajado/a a regressar e completar desafios diariamente	3,92
Os caminhos de aprendizagem IA adaptaram-se eficazmente ao meu desempenho	4,08

6.2.3 Análise Qualitativa

As respostas abertas revelaram pontos fortes e oportunidades de melhoria consistentes.

Pontos fortes. A explicação adaptativa após resposta errada foi apontada como melhor funcionalidade por quatro participantes; a intuitividade da interface por outros quatro; e a adaptabilidade da IA ao desempenho individual por três. Os desafios diários e a relevância das questões foram igualmente referidos positivamente.

Problemas técnicos. A mais grave foi uma *race condition* visual na remediação adaptativa: a resposta era marcada correta e, meio segundo depois, aparecia como errada. Foram também reportados a barra de pesquisa oculta durante a inserção de e-mail, respostas corretas marcadas intermitentemente como erradas, e questões repetidas em caminhos alternativos. Os dois participantes com pontuações mais baixas (P04, P05) apontaram ainda a confusão causada pela transição abrupta para um novo desafio após resposta errada, sem explicação, um comportamento que ocorre antes de a sessão de remediação adaptativa ser criada.

Perfis de afinidade tecnológica. A observação direta revelou uma segunda linha de segmentação, além do vínculo institucional (Tabela 6.2): P04 e P05, os dois *outliers* negativos, pertencem a um perfil de utilização tecnológica mínima, sugerindo que a sua pontuação reflecte não só o bug de transição identificado acima, mas também uma dificuldade de base com aprendizagem digital autónoma, precisamente o perfil que mais beneficiaria do Play4Change (retomada em §7.5).

Sugestões de melhoria. As mais recorrentes foram funcionalidades sociais (amigos, tabelas de classificação), diversificação de tópicos, maior concisão das explicações IA em mobile, e personalização geográfica das questões por localização.

Conclusão da avaliação. Os resultados SUS indicam uma usabilidade global consistentemente acima da média de referência da indústria (68,0), com a maioria dos participantes (sete em doze) a atingir o limiar de top 10% (≥ 80) reportado por Sauro/NNGroup [36]. Os dois *outliers* convergem num problema de comunicação de estado após erro, mitigável com uma transição animada, mas reflectem também uma barreira de entrada mais alta para o perfil de menor afinidade tecnológica.

Capítulo 7

Discussão

Este capítulo confronta os resultados com os objetivos propostos, identifica o que correu melhor e pior, posiciona o Play4Change face ao estado da arte, e aponta direções de trabalho futuro.

7.1 Objetivos Atingidos

Os cinco objetivos operacionais foram todos atingidos, com diferentes graus de exigência.

OBJ-1: Plataforma adaptativa DigComp 2.2

O sistema implementa inscrição com gating de pré-requisitos, entrega calendarizada de tarefas, remediação por dificuldade e escalamento para tutoria IA. O alinhamento com o DigComp 2.2 depende da curação humana dos tópicos, não de verificação automática. Na avaliação de usabilidade (Secção 6.2), sete dos doze participantes (58,3%) atingiram uma pontuação SUS de 80 ou superior, o limiar que Sauro reporta como correspondente ao top 10% de mais de 500 estudos de referência [36].

OBJ-2: Pipeline de geração por IA

O pipeline (*ingestion*, *chunking*, geração com *schema reminder*, deduplicação semântica) funciona de ponta a ponta e degrada graciosamente em caso de falha (circuit breaker, retry). As seis garantias de qualidade estão detalhadas no Apêndice C.

OBJ-3: Remediação adaptativa

O `ErrorPatternClassifier` classifica cada erro em três padrões e a reutilização semântica de ramos (§5.2.2.2) reduz custos de API. A limitação principal é a profundidade fixa (`MAX_STRUGGLE_DEPTH = 1`), analisada criticamente em §7.3.

OBJ-4: Tutoria com IA

A `ExplanationSession` (§5.2.2.3) cobre geração assíncrona, diálogo multi-turno e retorno ao fluxo principal. Foi a funcionalidade mais valorizada na avaliação qualitativa (§6.2.3).

OBJ-5: Segurança e qualidade

A segurança não foi o foco do projeto, mas dez vetores de ataque concretos foram endereçados (§5.3), três com exposição residual reconhecida; a suite de segurança valida SSRF, XSS, o fluxo de código de acesso e controlo de acesso ao Swagger. Não há pretensão de cobertura exaustiva.

7.2 O que Correu Bem

Arquitetura hexagonal

Modelar todas as dependências externas como interfaces (§4.1) foi a decisão mais rentável do projeto: torna a suite de testes rápida e determinística (§6.1), a ponto de permitir substituir o próprio relógio por um `ManualTimeMeter` (§6.1) e tornar 100% deterministas até os testes de janelas temporais de *rate limiting*.

Reutilização semântica de ramos

A reutilização por similaridade coseno (§5.2.2.2, Tabela 5.3) reduz o custo de API à medida que a base de ramos cresce, sem intervenção humana, o contributo técnico mais original do sistema.

Resiliência do pipeline de IA

Circuit breaker, degradação graciosa e *fail-safe* tornam o sistema tolerante a falhas intermitentes do Mistral, com visibilidade via o alerta `AiCircuitBreakerOpen`.

Autenticação sem senha

O fluxo de código de acesso por email elimina a gestão de passwords e cobre as ameaças relevantes ao modelo de autenticação (§5.1.2).

7.3 Limitações e Dificuldades

Conformidade de schema do LLM

O Mistral não garante JSON válido em todas as chamadas. O *schema reminder* mitiga, não elimina, a falha; sem dados de produção não é possível quantificar a taxa real de ocorrência.

Profundidade fixa de struggle

A escolha de `MAX_STRUGGLE_DEPTH = 1`, justificada na Secção 5.2.2.2, deixa tópicos avançados sem patamares intermédios antes do escalamento para tutoria; o compromisso identificado fica por endereçar em trabalho futuro (§7.5).

Falta de dados de produção

O sistema instrumenta 20 métricas e oito alertas, mas a validação assentou em 12

participantes num ambiente controlado. Latências reais, distribuição de estratégias de reutilização e taxa de deduplicação sob carga permanecem por medir.

Fornecedor de IA único

O sistema está acoplado ao Mistral; a abstração LangChain4j isola a lógica de negócio, mas os prompts e os *embeddings* estão calibrados a este modelo; migrar exigiria recalibração e *re-embedding* de todo o catálogo.

Contexto *Report* por integrar

O mecanismo de sinalização de tarefas incorretas pelo Learner (revisão do Administrador, com correção ou dispensa) está implementado no servidor e no painel web, mas não ligado a nenhum cliente: restam por decidir a política de suspensão de conteúdo, as salvaguardas contra falsos alarmes, e o tratamento de instâncias já entregues.

7.4 Comparação com o Estado da Arte

O Play4Change cruza três linhas do Capítulo 2: plataformas gamificadas, sistemas de tutoria inteligente (ITS) e geração de conteúdo por LLM.

Face às plataformas gamificadas

(Tabela 2.1, §2.5) Nenhuma gera conteúdo próprio a partir de fontes documentais de uma instituição, mas todas oferecem hoje mais funcionalidades sociais e de retenção, a lacuna mais relevante para o trabalho futuro.

Face aos ITS clássicos

(Cognitive Tutor e sistemas equivalentes [9]) Estes usam modelos de utilizador explícitos como o *Bayesian Knowledge Tracing* [10], estatisticamente mais precisos, mas só depois de calibrados sobre um histórico de uso suficiente por domínio. O Play4Change segue o caminho oposto, com vantagens próprias: o `ErrorPatternClassifier` é determinístico e explicável (a razão de cada classificação é rastreável, não uma probabilidade aprendida), não chama o modelo de IA nem tem custo de inferência, e funciona com precisão total desde o primeiro Learner e o primeiro tópico, em qualquer domínio, sem período de calibração, ao custo de uma resolução diagnóstica mais grosseira do que a de um modelo probabilístico já maduro.

Face ao uso direto de LLM para tutoria

(ChatGPT, Claude) É flexível mas sem percurso definido, gamificação ou isolamento de input. O Play4Change estrutura a progressão (tarefa → struggle → explicação → re-tentativa), gamifica, garante qualidade por schema e deduplicação, e isola o input do utilizador em mensagens tipadas.

7.5 Trabalho Futuro

Duas direções têm prioridade estratégica:

Funcionalidades sociais e de retenção

Amigos, tabelas de classificação, conquistas sobre a infraestrutura de `micro_competences` já existente, e notificações comportamentais, respondem à sugestão mais recorrente da avaliação de usabilidade e à declaração de envolvimento mais baixa (regresso diário, 3,92/5).

Reconhecimento formal via Open Badges e créditos ECTS

Tornaria o sistema de crachás atual, hoje interno e não portátil, exportável e verificável externamente, com potencial para micro-créditos ECTS (*European Credit Transfer and Accumulation System*) na rede U!REKA.

Duas direções estratégicas, não necessariamente exclusivas, ficam em aberto: aprofundar a vertente universitária para o público tecnologicamente confortável, ou investir em acompanhamento humano para o perfil de menor afinidade digital.

Referências

- [1] Riina Vuorikari, Stefano Kluzer, and Yves Punie. DigComp 2.2: The Digital Competence Framework for Citizens — With new examples of knowledge, skills and attitudes. EUR 31006 EN, Publications Office of the European Union, Luxembourg, 2022. JRC128415.
- [2] European Parliament and Council of the European Union. Decision (EU) 2022/2481 of the European Parliament and of the Council of 14 December 2022 establishing the Digital Decade Policy Programme 2030. Official Journal of the European Union, L 323, 2022. Sets the target of at least 80% of EU adults with basic digital skills by 2030, building on the 2030 Digital Compass (COM(2021) 118 final).
- [3] Benjamin S. Bloom. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6):4–16, 1984.
- [4] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [6] Jay Cross. *Informal Learning: Rediscovering the Natural Pathways That Inspire Innovation and Performance*. Pfeiffer, San Francisco, CA, 2007.
- [7] Burrhus F. Skinner. Teaching machines. *Science*, 128(3330):969–977, 1958.
- [8] John R. Anderson, C. Franklin Boyle, and Brian J. Reiser. Intelligent tutoring systems. *Science*, 228(4698):456–462, 1985.
- [9] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011.

- [10] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1994.
- [11] Chris Piech, Jonathan Spencer, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [12] TechCrunch. Duolingo launches 148 courses created with AI after sharing plans to replace contractors with AI. TechCrunch, 2025. [Online; accessed 2026].
- [13] Khan Academy. Khanmigo: Khan Academy’s AI-powered teaching assistant & tutor. Khan Academy, 2025. [Online; accessed 2026].
- [14] Coursera. New AI-powered innovations on Coursera: Role play for authors, course builder for partners, skills tracks for enterprise, and more. Coursera Blog, 2024. [Online; accessed 2026].
- [15] Rishi Bommasani, Drew A. Hudson, Ehsan Aditi, et al. On the opportunities and risks of foundation models. Technical report, Stanford University, 2021.
- [16] John Hattie. *Visible Learning: A Synthesis of Over 800 Meta-Analyses Relating to Achievement*. Routledge, London, 2009.
- [17] Adrian Simpson. The misdirection of public policy: Comparing and combining standardised effect sizes. *Journal of Education Policy*, 32(4):450–466, 2017.
- [18] John Sweller. Cognitive load theory, learning difficulty, and instructional design. *Learning and Instruction*, 4(4):295–312, 1994.
- [19] Helen Colley, Phil Hodgkinson, and Janice Malcolm. Informality and formality in learning: A report for the learning and skills research centre. *Learning and Skills Research Centre*, 2003.
- [20] Malcolm S. Knowles. *Andragogy in Action: Applying Modern Principles of Adult Learning*. Jossey-Bass, San Francisco, 1984.
- [21] John Traxler. Defining, discussing and evaluating mobile learning: The moving finger writes and having writ. *International Review of Research in Open and Distributed Learning*, 8(2), 2007.
- [22] Diana Laurillard. Pedagogical forms of mobile learning: Framing research questions. *Mobile Learning: Towards a Research Agenda*, pages 153–175, 2007.

- [23] International Telecommunication Union. Measuring digital development: Facts and figures 2023. Technical report, ITU, Geneva, 2023.
- [24] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint*, arXiv:2312.10997, 2024.
- [25] Ian Sommerville. *Software Engineering*. Pearson, Harlow, 10th edition, 2016.
- [26] Donald A. Norman. *The Design of Everyday Things*. Basic Books, New York, 1988.
- [27] Hermann Ebbinghaus. *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Duncker & Humblot, Leipzig, 1885. Translated as *Memory: A Contribution to Experimental Psychology*, 1913.
- [28] Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- [29] Edward L. Deci and Richard M. Ryan. *Intrinsic and Extrinsic Motivations: Classic Definitions and New Directions*, volume 25. 2000.
- [30] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, Boston, 4th edition, 2021.
- [31] Ivar Jacobson, Magnus Christerson, Patrik Jonsson, and Gunnar Övergaard. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, Wokingham, 1992.
- [32] Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *IEEE Symposium on Security and Privacy*, pages 553–567, 2012.
- [33] Unstructured Technologies. Unstructured: Open source library and API for document extraction, 2023. [Online; accessed 2025].
- [34] Microsoft Corporation. Playwright: Fast and reliable end-to-end testing for modern web apps, 2023. [Online; accessed 2025].
- [35] John Brooke. SUS: A ‘quick and dirty’ usability scale. pages 189–194, 1996.
- [36] Page Laubheimer. Beyond the NPS: Measuring perceived usability with the SUS, NASA-TLX, and the single ease question after tasks and usability tests. Nielsen Norman Group, 2018. [Online; accessed 2026].

Apêndice A

Exemplos de Prompts Completos

Este apêndice reproduz, na íntegra e sem alterações, os *templates* de *prompt* definidos no módulo `ai-agent` (pacote com `.ureka.play4change.prompt`) e efetivamente enviados ao modelo `mistral-small-latest` através do `LangChain4j`. Os campos entre `$` são interpolados em *run-time* a partir do contexto de domínio (tópico, módulo, sessão de remediação adaptativa, etc.). Todos os *prompts* de geração de conteúdo obrigam o modelo a responder exclusivamente em JSON, sem texto envolvente, para permitir *parsing* determinístico pelo `OptionsJsonParser`.

A.1 Prompt de Geração de Tópicos e Desafios

Utilizado pelo `LangChain4jTaskGenerationAdapter` sempre que é necessário gerar tarefas de escolha múltipla: quer na ingestão inicial de um tópico (`TaskGenerationOrchestrator`), quer na geração de variantes por idioma (`LanguageGenerationAdapter`), quer na geração de instâncias distratoras em lote (`BatchInstanceAdapter`).

System Message

You are an expert educational content designer specialising in gamified learning experiences. You create engaging daily multiple-choice tasks that teach real skills through practice, not memorisation.

LANGUAGE: You MUST generate ALL text content in `$language`.

This is mandatory. Do not mix languages. Do not respond in English unless `$language` is English.

RULES:

- Each task must be completable in 5-15 minutes
- Tasks must be practical and actionable, not theoretical
- Difficulty must match the audience level exactly
- Hints must guide without giving away the answer

- Every task must directly advance the module objective
- Tasks must be distinct - no semantic overlap with existing content
- Always generate exactly 4 options per task
- The correct answer must always be at index 0 in the options array
(the system will shuffle before showing to the user)
- Wrong options must be plausible - not obviously incorrect

OUTPUT FORMAT: Respond with a valid JSON array only. No preamble. No explanation.

Schema:

```
[
  {
    "title": "string (max 60 chars)",
    "description": "string (max 300 chars - frame it as a question or scenario)",
    "hint": "string (max 150 chars)",
    "pointsReward": number (10-100, higher for harder tasks),
    "options": ["correct answer", "wrong option 1", "wrong option 2", "wrong option 3"],
    "correctAnswerIndex": 0
  }
]
```

User Message

Generate \${request.taskCount} multiple-choice learning tasks for the following context:

Subject Domain: \${request.subjectDomain}

Module Objective: \${request.moduleObjective}

Audience Level: \${request.audienceLevel.toPromptDescription()}

Language: \${request.language}

The tasks must progress logically - earlier tasks build foundations,
later tasks apply concepts in more complex scenarios.

Important: N tasks already exist for this module. Generate semantically
distinct tasks that complement the existing content.

[linha condicional, omitida quando não existem tarefas prévias no módulo]

Remember: always put the correct answer at index 0 in the options array.

Generate the JSON array now:

O nível de audiência (AudienceLevel) é traduzido para uma descrição textual antes de

ser injetado no *prompt*:

```
BEGINNER      -> "Beginner - assumes no prior knowledge, uses simple
                  language, concrete examples"
INTERMEDIATE  -> "Intermediate - assumes basic familiarity, introduces
                  complexity, some abstraction"
ADVANCED      -> "Advanced - assumes solid foundation, focuses on nuance,
                  edge cases, and depth"
```

Reforço de Esquema (retry)

Quando a resposta do modelo é incompleta ou não corresponde ao número de tarefas pedido, o Resilience4j `retry` reenvia o pedido com um reforço adicional do esquema:

```
Your previous response was incomplete or malformed.
Please respond with EXACTLY $taskCount tasks in the JSON array schema above.
Every task must have: title, description, hint, pointsReward, options (4 items),
and correctAnswerIndex set to 0. No extra fields. Valid JSON only.
```

A.2 Prompt de Análise de Dificuldade e Geração de Subtarefas Adaptativas

Utilizado pelo `HandleStruggleService` quando um aprendiz falha repetidamente uma tarefa. Ao contrário do *prompt* de geração base, este inclui o padrão de erro diagnosticado (`ErrorPattern`), permitindo que o modelo gere um percurso de recuperação escalonado em vez de tarefas genéricas sobre o mesmo tópico.

System Message

```
You are an expert adaptive learning coach. A student is struggling with
a specific task in their learning journey. Your job is to create a short
sequence of simpler subtasks that scaffold the student back to success.
```

PRINCIPLES:

- Diagnose the root cause of the struggle from the error pattern
- Start with the simplest possible related concept
- Each subtask builds on the previous one
- The final subtask should bring the student back to the original task level
- Be encouraging - frame tasks as achievable steps, not remediation
- Keep each subtask completable in 3-10 minutes

OUTPUT FORMAT: Respond with a valid JSON array only. No preamble. No explanation.

Schema:

```
[
  {
    "title": "string (max 60 chars)",
    "description": "string (max 300 chars)",
    "hint": "string (max 150 chars)",
    "pointsReward": number (5-50, lower than main tasks to reflect smaller scope),
    "options": ["string", "string", "string", "string"],
    "correctAnswerIndex": number (0-based index into options array)
  }
]
```

User Message

A student is struggling with the following task after `${context.attemptCount}` attempts:

Task: `${context.taskDescription}`

Subject Domain: `${context.subjectDomain}`

Module Objective: `${context.moduleObjective}`

Student Level: `${context.audienceLevel.toPromptDescription()}`

Language: `${context.language}`

Diagnosed Struggle Type: `${context.errorPattern.toPromptDescription()}`

Generate `$subtaskCount` adaptive subtasks that:

1. Address the specific struggle type diagnosed above
2. Start simpler than the original task
3. Progressively build back toward the original task level
4. Are encouraging and reframe the struggle as a learning opportunity

Generate the JSON array now:

Cada `ErrorPattern` é traduzido numa instrução específica de diagnóstico antes de ser injetado no *prompt*:

CONCEPTUAL_MISUNDERSTANDING -> "The student misunderstands the core concept.

Start by re-explaining it differently."

PROCEDURAL_ERROR -> "The student understands the concept but

applies the wrong steps. Focus on procedure."

KNOWLEDGE_GAP	-> "The student is missing prerequisite knowledge. Fill the gap before returning to the task."
UNCLEAR_INSTRUCTIONS	-> "The task instructions were unclear. Provide clearer, more explicit guidance."
TIME_PRESSURE	-> "The student rushed and made errors under time pressure. Use paced, step-by-step practice tasks."
UNKNOWN	-> "The struggle cause is unclear. Provide general scaffolding from basic to intermediate."

A.3 Prompt de Explicação Holística e Conversação

Utilizado pelo `ExplanationService` quando um aprendiz esgota todos os níveis de profundidade do percurso adaptativo sem resolver a dificuldade. Divide-se em dois modos: a explicação inicial (síntese de todas as tentativas) e as respostas conversacionais subsequentes, caso o aprendiz continue confuso.

System Message: Explicação Inicial

You are a patient and empathetic learning coach helping a student who has struggled repeatedly with the same concept. They have now exhausted all their practice attempts and need a clear, holistic explanation.

YOUR GOAL:

- Synthesise what went wrong across all their attempts
- Explain the underlying concept clearly and simply
- Use concrete examples relevant to the subject domain
- Be encouraging - frame struggles as a normal part of learning
- Keep the explanation concise: 3-5 paragraphs maximum
- Write in plain prose - no JSON, no bullet lists, no headers
- Match the learner's language exactly

User Message: Explicação Inicial

A student in a `${context.audienceLevel.toLabel()}` course on "`${context.subjectDomain}`" has struggled with the following task after exhausting their practice attempts:

Task: `${context.taskDescription}`

Module objective: \${context.moduleObjective}
Struggle type: \${context.errorPattern.toLabel()}
Language: \${context.language}

Here is their struggle history:

Session \${depth} (\${errorPattern.toLabel()}): \${taskTitles joined by ", "}
[uma linha por sessão de remediação adaptativa anterior]

Write a clear, encouraging explanation that helps them finally understand this concept.

Respond in \${context.language}. Plain prose only.

System Message: Réplica Conversacional

You are a patient learning coach in an ongoing conversation with a confused student. They have already received an initial explanation of a concept they struggled with. Your role now is to answer their specific questions and clear up remaining confusion.

PRINCIPLES:

- Address their exact question directly
- Use simple language and concrete examples
- Keep replies focused: 2-4 sentences unless the question demands more
- Be warm and encouraging
- Write in the same language as the student
- Plain prose only - no JSON, no bullet lists

O primeiro turno da conversa é sempre um bloco de contexto gerado a partir de dados confiáveis (provenientes da base de dados), nunca de conteúdo fornecido pelo utilizador, uma separação deliberada que impede que uma futura mensagem maliciosa do aprendente altere o enquadramento da conversa (ver Secção 5.3 sobre defesas contra *prompt injection*):

Subject: \${context.subjectDomain}
Original task: \${context.taskDescription}